
Helios

**Lumiere Cinema
Software Architecture Document**

Version 2.1

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

Revision History

Date	Version	Description	Author
18/06/2025	1.0	Write Introduction	Ngô Hồng Thanh
		Write Goals and Constraints	Vương Ngũ Tín Thành
		Write Logical View (overview)	Nguyễn Thiên Nhã Trân
19/06/2025	1.1	Write Administrator service	Phan Nhựt Anh
		Write Authentication service , User service , Movie service , Branch service , Ticket service , Report service	Ngô Hồng Thanh
		Write Component: Hooks	Phan Nhựt Anh
20/06/2025	1.2	Write UI Overview	Phan Nhựt Anh
		Write Movie navigation , User profile , Dashboard layout	Ngô Hồng Thanh
		Write Ticket booking flow , Authentication layout	Nguyễn Thiên Nhã Trân
23/06/2025	1.3	Write Route Overview	Tạ Thiên Lâm
		Write Authentication route	Ngô Hồng Thanh
		Write User route , Move route , Branch route , Report route , Administration route	Tạ Thiên Lâm
		Write Ticket route	Vương Ngũ Tín Thành
		Write Controller Overview	Vương Ngũ Tín Thành
		Write Authentication controller , User controller	Ngô Hồng Thanh
		Write Movie controller , Branch controller , Branch controller , Report controller , Chatbot controller , Administrator controller .	Tạ Thiên Lâm
		Write Ticket controller	Nguyễn Thiên Nhã Trân
26/06/2025	1.4	Write Component: Models	Vương Ngũ Tín Thành
		Write Component: Database	Vương Ngũ Tín Thành
		Write Component: Large Language Model	Tạ Thiên Lâm
28/06/2025	1.5	Draw Deployment Graph	Nguyễn Thiên Nhã Trân
		Write Client-side & Server-side	Vương Ngũ Tín Thành
		Write Third-Party Services Integration	Ngô Hồng Thanh
		Write Implementation view: Back-end	Tạ Thiên Lâm
		Write Implementation view: Front-end	Phan Nhựt Anh

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

Date	Version	Description	Author
17/07/2025	2.0	Revise Authentication route : Separate staff's login and reset password route from customer's ones Revise Authentication controller : Separate Forgot password and Reset password	Vương Ngũ Tín Thành
		Revise User routes , Movie routes , Branch routes , Ticket routes and Administrator routes . Add new routes: QR routes and Chatbot routes Revise controller	Tạ Thiên Lâm
25/07/2025	2.1	Revise Authentication service , User service , Branch service , Ticket service , Report service , Administration service Revise Authentication logic , Ticket logic , Administration logic	Ngô Hồng Thanh

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

Table of Contents

- 1. Introduction..... 4**
 - 1.1 Purpose..... 4
 - 1.2 Scope..... 4
 - 1.3 Overview..... 4
- 2. Architectural Goals and Constraints.....4**
 - 2.1 Architectural goals..... 4
 - 2.2 Architectural constraints..... 5
- 3. Use-Case Model..... 6**
- 4. Logical View.....7**
 - 4.1 Component: Services..... 7
 - 4.2 Component: Hooks..... 11
 - 4.3 Component: User Interface (UI)..... 15
 - 4.4 Component: Routers..... 20
 - 4.5 Component: Controllers..... 26
 - 4.6 Component: Model..... 30
 - 4.7 Component: Database..... 36
 - 4.8 Component: Large Language Model..... 37
- 5. Deployment..... 38**
 - 5.1 Client-side..... 38
 - 5.2 Server-side..... 38
 - 5.3 Third-Party Services Integration - Large Language Model (LLM)..... 38
- 6. Implementation View..... 39**
 - 6.1 Front-end..... 39
 - 6.2 Back-end..... 40

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

Software Architecture Document

1. Introduction

The Software Architecture Document provides a comprehensive overview of the system's architectural design. It serves as a blueprint for the development team, stakeholders, and future maintainers, offering insights into the high-level structure, major components, their interactions, and key architectural decisions.

1.1 Purpose

The purpose of this document is to describe the software architecture of the Lumiere Cinema system. It aims to ensure that developers, testers, designers, and project managers, have a shared understanding of the system architecture. This document will support decision-making during development, guide implementation, and facilitate communication among team members.

1.2 Scope

This Software Architecture Document outlines the overall architecture of the Lumiere Cinema system, focusing on its major components, modules, interfaces, and the way they interact. It explains the structural composition of the system, the core design principles, and the rationale behind the selection of architectural styles and patterns.

1.3 Overview

The following sections provide a comprehensive breakdown of the architecture of the Lumiere Cinema system. The document is organized as follows:

- **Introduction:** Provides an introduction to the Software Architecture Document, its purpose, scope and an overview of the content.
- **Architectural Goals and Constraints:** Identifies the key goals and limitations that shaped architectural decisions.
- **Use-Case Model:** Presents the architectural perspective from the standpoint of system use cases and scenarios.
- **Logical View:** Details the logical organization of the system, emphasizing the division of responsibilities among different components.
- **Deployment:** Demonstrates how software components are deployed onto the physical hardware environment.
- **Implementation View:** Describes how the components are implemented and organized in the codebase.

2. Architectural Goals and Constraints

This section describes the primary goals and constraints that significantly impact the architectural design of the Lumiere Cinema system. These are derived from the non-functional requirements and the project's development environment.

2.1 Architectural goals

Architectural goals are the critical non-functional requirements that the system must achieve, having a major influence on how the system is designed.

- High performance:
 - + Ensure that the system responds to user actions (e.g., login, movie search, seat selection, payment) within 3 seconds for 95% of cases.
 - + Maintain system stability and responsiveness even during peak traffic periods, especially during new movie releases or prime booking hours.
- Scalability:
 - + The architecture must be easily scalable to accommodate future increases in the number of users, cinema branches, and transaction volumes without significant structural changes.
 - + Support the addition of new features without substantially impacting existing functionalities.
- Reliability & Availability:
 - + The system must exhibit good fault tolerance, providing clear error messages and recovering gracefully in case of failures.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

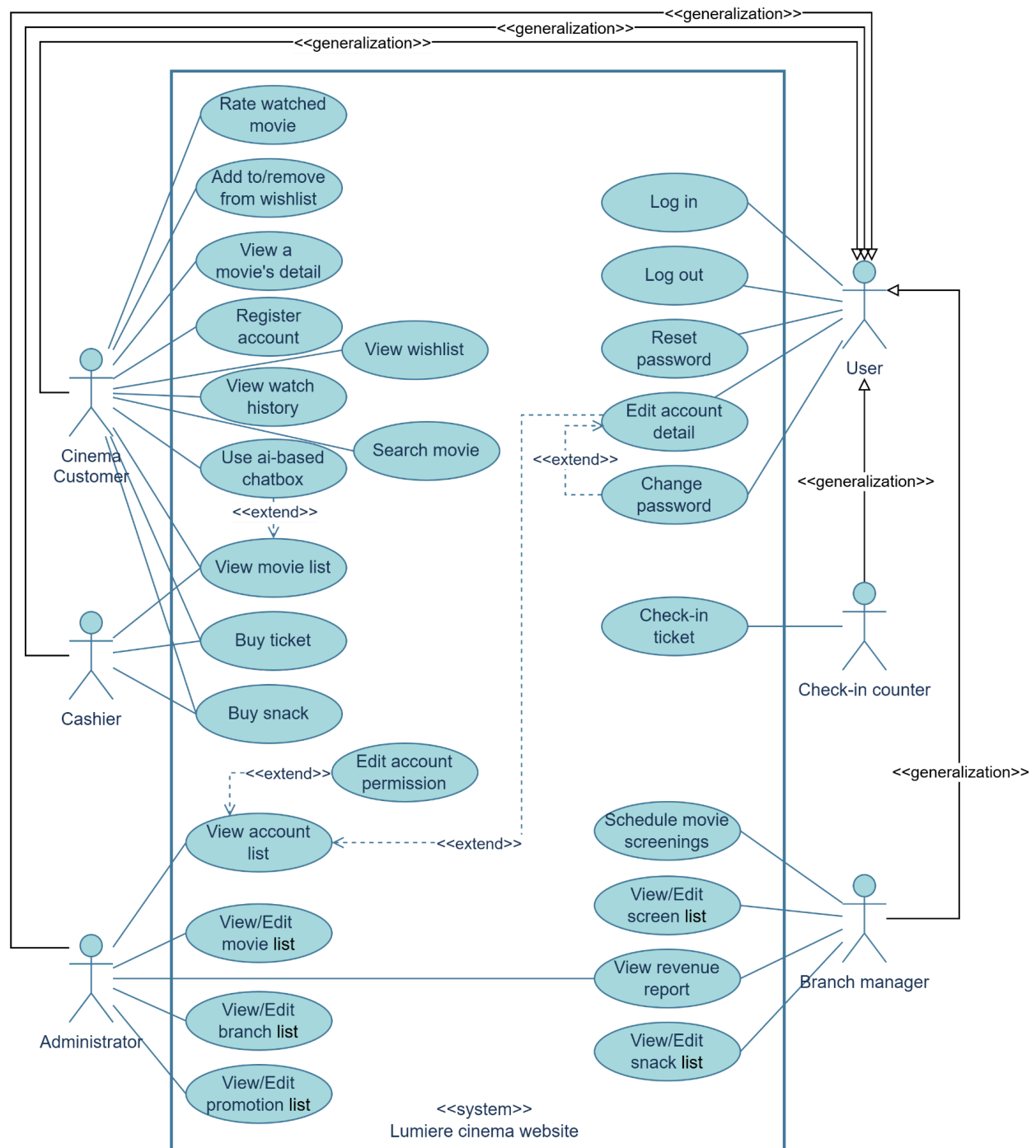
- + Guarantee data integrity and security, allowing changes only through authorized channels.
- Security:
 - + Implement robust role-based access control (RBAC) for users, staff, and administrators.
 - + Log all significant user interactions and system activities for auditing purposes and anomaly detection.
- Usability:
 - + Ensure the user interface is responsive and optimized for various devices and screen sizes (desktop, laptop, tablet, smartphone).
 - + Support popular modern web browsers (Chrome, Firefox, Edge).
 - + Provide a clear, intuitive, and easy-to-use interface for all user types.
- Maintainability:
 - + The architecture should be modular, easy to understand, modify, and upgrade.
 - + The codebase must adhere to good design principles and be well-documented.

2.2 Architectural constraints

- Language and technology stack:
 - + The website is built by using HTML, CSS, JavaScript and NoSQL.
 - + This project uses ReactJS and TailwindCSS for front-end; Express and NodeJS for back-end; MongoDB and Mongoose for Database management.
- Deployment environment: The primary application will be a web application, requiring compatibility with major web browsers.
- Development tools: Jira for project management, GitHub for source code version control, Google Drive for documentation storage, and Figma for software prototyping.

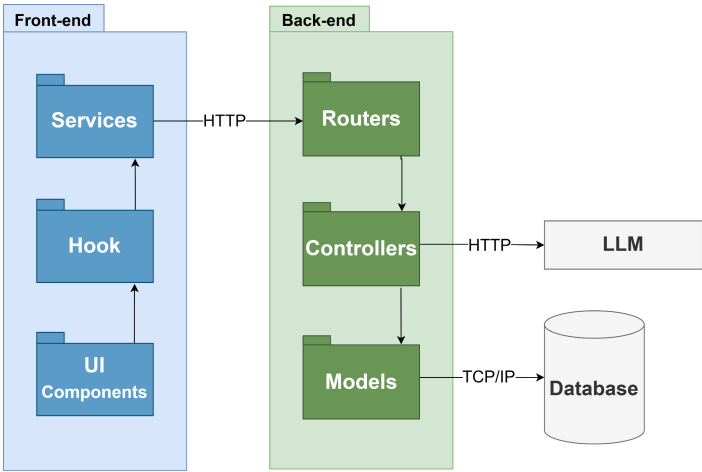
Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

3. Use-Case Model



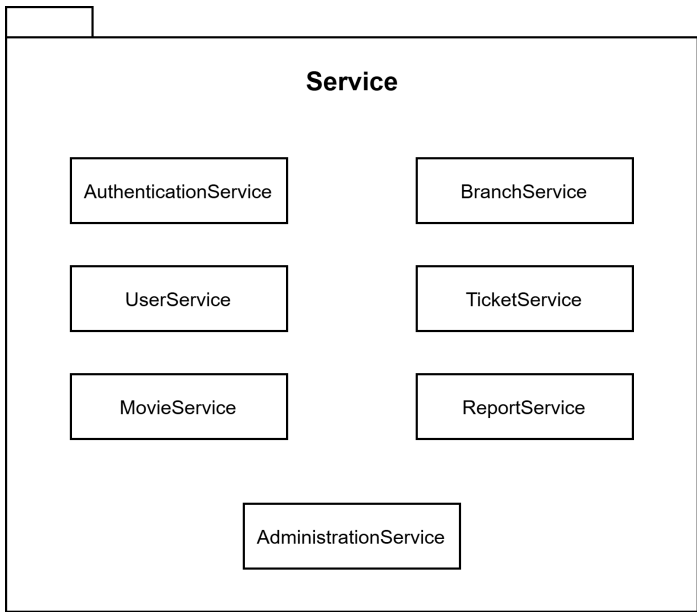
Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4. Logical View



The architecture of this software references two popular models, corresponding with the programming framework. The front-end is structured based on the three-layer architecture in ReactJS, including services, hooks and UI components. The back-end is structured based on the MVC architecture, including routers, controllers and models. The front-end and back-end communicate with each other through the front-end's service and back-end's routers.

4.1 Component: Services



In the front-end architecture of the Lumiere Cinema, services are dedicated modules that encapsulate communication with back-end APIs. Each service is responsible for sending HTTP requests (GET, POST, PUT/PATCH, DELETE) and receiving responses, without handling any UI state or rendering logic. Services act as a separation layer between the user interface (components, hooks) and the back-end. This abstraction ensures that:

- API logic is centralized, reusable, and easy to test.
- Hooks can focus on managing UI state, side effects, and error handling.
- Changes to API endpoints or formats only affect the services, not the entire application.

Each service corresponds to a specific domain of functionality and exposes functions that are called internally by

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

custom React hooks, such as useAuthentication, useUser, useTicket,...

4.1.1 Authentication service

AuthenticationService
+ login() + register() + resetPassword() + changePassword() + logout() + activateAccount() + forgotPassword() + staffLogin() + staffForgotPassword()

Authentication Service handles communication with the back-end for user authentication features.

Functions:

login(): Sends a POST request to the back-end with login credentials and receives an auth token if valid.

register(): Sends a POST request to register a new user account with name, email, and password.

resetPassword(): Sends a POST request for a password reset, which triggers an email with reset instructions.

changePassword(): Sends a POST request to update the password after verifying the current one on the server.

logout(): Logs the user out of the system.

activateAccount(): Sends a POST request to the back-end to activate a user account using a token received via email.

forgotPassword(): Sends a POST request to initiate the password reset process for a user who has forgotten their password.

staffLogin(): Sends a POST request to the back-end with staff credentials to authenticate staff users.

staffForgotPassword(): Sends a POST request to trigger a password reset flow for staff accounts, similar to **forgotPassword()**, but scoped for staff roles.

4.1.2 User service

UserService
+ getProfile() + updateProfile() + getWatchHistory() + getWishlist() + addToWishlist() + removeFromWishlist() + rateMovie() + getRatings() + removeFromWatchHistory() + getUserTickets()

User Service provides methods for interacting with back-end APIs related to user profile, watch history, wishlist, and movie ratings.

Functions:

getProfile(): Sends a GET request to the back-end to retrieve the current user's profile information.

updateProfile(): Sends a PUT/PATCH request to update profile fields (e.g. name, avatar) on the server.

getWatchHistory(): Requests watch history data from the back-end tied to the current user.

getWishlist(): Fetches the current user's wishlist by sending a GET request to the server.

addToWishlist(): Sends a POST request to add a movie to the user's wishlist on the server.

removeFromWishlist(): Sends a DELETE request to remove a movie from the wishlist in the back-end.

rateMovie(): Sends a POST request to submit a rating for a movie, linked to the current user.

getRatings(): Sends a GET request to retrieve the user's rating(s). May be filtered by movie.

removeFromWatchHistory(): Sends a DELETE request to remove a specific movie or all entries from the user's watch history.

getUserTickets(): Sends a GET request to retrieve all movie and snack tickets purchased by the current user.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.1.3 Movie service

MovieService
+ getNowShowing() + getUpcoming() + getMovieDetail() + getShowtimes() + searchMovies() + getMovieRatings() + getAllMovies() + addMovie() + updateMovie() + deleteMovie()

Movie Service exposes functions to fetch and filter movie-related data from the back-end, including listings, search results, and showtimes.

Functions:

getNowShowing(): Sends a GET request to retrieve currently playing movies from the server.

getUpcoming(): Fetches a list of upcoming movies by querying the back-end.

getMovieDetail(): Sends a GET request to retrieve detailed movie data by ID.

getShowtimes(): Fetches showtimes from the back-end for a given movie.

searchMovies(): Sends a GET request to the back-end to find movies by keyword.

getMovieRatings(): Sends a GET request to retrieve the rating for a movie.

getAllMovies(): Sends a GET request to retrieve all movies (including inactive ones) for admin management.

addMovie(): Sends a POST request to add a new movie.

updateMovie(): Sends a PUT/PATCH request to update details of an existing movie.

deleteMovie(): Sends a DELETE request to remove or deactivate a movie from the system.

4.1.4 Branch service

BranchService
+ getBranches() + getBranchDetails() + getScreens() + getScreenDetail() + getSchedules() + getSnacks() + scheduleMovie() + addScreen() + addSnack() + updateScreen() + updateSnack() + updateSchedule() + removeScreen() + removeSnack() + removeSchedule()

Branch Service manages back-end interaction for branch-specific features such as screens, schedules, snack menus, movies, and staff.

Functions:

getBranches(): Sends a GET request to retrieve all branches in the cinema system.

getBranchDetails(): Sends a GET request to fetch detailed information about a specific branch

getScreens(): Fetches a list of screens from the specified branch via API.

getScreenDetail(): Sends a GET request to retrieve detailed information for a specific screen within a branch for management purposes

getSchedules(): Sends a request to get movie showtimes at the branch for a given date.

getSnacks(): Retrieves available snacks and combos at a given branch from the back-end.

scheduleMovie(): Sends a POST request to create a schedule/showtime for a movie on a given screen.

addScreen(): Sends a POST request to create a new cinema screen within a specific branch.

addSnack(): Sends a POST request to add a new snack item to a specific branch's snack list.

updateScreen(): Sends a PUT/PATCH request to update details of an existing screen within a specific branch.

updateSnack(): Sends a PUT/PATCH request to update details of an existing snack item for a specific branch.

updateSchedule(): Sends a PUT/PATCH request to update the movie screening at a specific screen and time for a branch.

removeScreen(): Sends a DELETE request to remove a specific screen from a specific branch.

removeSnack(): Sends a DELETE request to remove a snack item from a specific branch's list.

removeSchedule(): Sends a DELETE request to cancel an existing movie schedule for a specific branch.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.1.5 Ticket service

TicketService
+ getSchedulesByBranch() + getSeatMapBySchedule() + createTicket() + calculateDiscountedTotal() + holdSeats() + manageSeatHold() + bulkReleaseSeatHolds() + cleanupExpiredHolds() + getAllMovieTickets() + getMovieTicketDetails() + getSnacksByBranch() + getAllSnackTickets() + getSnackTicketDetails() + getCacheStats() + cleanupCache() + preloadCache() + updateMovieTicket() + deleteMovieTicket() + updateSnackTicket() + deleteSnackTicket() + checkin()

Ticket Service handles back-end communication for the ticket booking process including seat availability, promotions, and ticket creation.

Functions:

getSchedulesByBranch(): Sends a GET request to fetch all movie schedules for a specific branch.

getSeatMapBySchedule(): Sends a GET request to retrieve the seating layout and availability for a selected movie schedule.

createTicket(): Sends a POST request to create and store a new ticket.

calculateDiscountedTotal(): Sends a POST request with cart details and promotion codes to calculate the discounts.

holdSeats(): Sends a POST request to temporarily hold selected seats for a short duration to prevent double booking.

manageSeatHold(): Sends a PATCH request to refresh, extend, or update existing seat holds during the booking process.

bulkReleaseSeatHolds(): Sends a DELETE request to release multiple held seats.

cleanupExpiredHolds(): Sends a POST request to clear all expired seat holds from the system.

getAllMovieTickets(): Sends a GET request to retrieve a list of all movie tickets for an admin query.

getMovieTicketDetails(): Sends a GET request to retrieve full details of a specific movie ticket.

getSnacksByBranch(): Sends a GET request to retrieve snack options and combos available at a particular branch for booking.

getAllSnackTickets(): Sends a GET request to retrieve all snack tickets.

getSnackTicketDetails(): Sends a GET request to retrieve full details of a specific snack ticket.

getCacheStats(): Sends a GET request to retrieve current statistics about cache usage for performance and monitoring.

cleanupCache(): Sends a POST request to clear expired or unused data from the ticket-related cache.

preloadCache(): Sends a POST request to pre-warm or pre-fetch cache data.

updateMovieTicket(): Sends a PATCH request to update ticket info, such as status, by authorized personnel.

deleteMovieTicket(): Sends a DELETE request to cancel or remove a movie ticket logically (admin use).

updateSnackTicket(): Sends a PATCH request to update status or details of a snack ticket (admin use).

deleteSnackTicket(): Sends a DELETE request to cancel or logically delete a snack ticket (admin use).

checkin(): Sends a PATCH request to mark a ticket as checked in at the cinema.

4.1.6 Report service

ReportService
+ getAvailableBranches() + getBranch() + getRevenueSummary()

Report Service handles communication with back-end reporting endpoints. Provides revenue data, supports export operations, and fetches available filtering options (branches, movies).

Functions:

getAvailableBranches(): Sends a GET request to retrieve branch options available for reporting filters.

getBranch(): Sends a GET request to retrieve detailed reporting data for a specific branch

getRevenueSummary(): Sends a GET request to fetch a summary of total revenue

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.1.7 Administration service

AdministrationService
+ getAllUsers() + getUserDetail() + addUser() + updateUserProfile() + deleteUser() + updateUserPermissions() + updateUserStatus() + getAllBranches() + addBranch() + updateBranch() + removeBranch() + updateBranchStatus()

Administration Service is responsible for handling back-end API communication related to system-wide administrative operations. This includes user account management, schedules, branches, staff, movies, snacks, and promotions. This service is exclusively used by administrator-level hooks like useAdministrator.

Function:

getAllUsers(): Sends a GET request to retrieve all user accounts in the system for admin management.

getUserDetail(): Fetches detailed profile data of a specific user account by ID.

addUser(): Sends a POST request to add a new user account.

updateUserProfile(): Sends a PATCH request to update the profile information of a user.

deleteUser(): Sends a DELETE request to remove a user account from the system.

updateUserPermissions(): Sends a POST request to assign or remove roles from a specific user account.

updateUserStatus(): Sends a PATCH request to update user status.

getAllBranches(): Sends a GET request to retrieve a full list of branches managed by the system.

addBranch(): Sends a POST request to add a new cinema branch to the system.

updateBranch(): Sends a PUT/PATCH request to update all high-level details of an existing cinema branch.

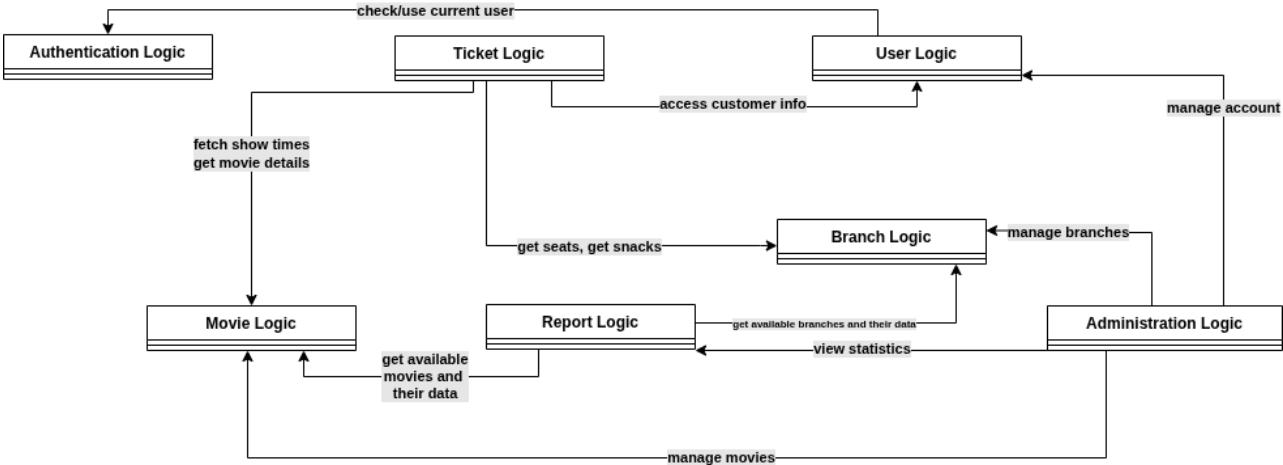
removeBranch(): Sends a DELETE request to remove a cinema branch from the system.

updateBranchStatus(): Sends a PATCH request to update branch status.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.2 Component: Hooks

The Hook layer in the front-end serves as an intermediary between the user interface (UI) and service layers. Each custom hook is organized by domain (e.g., movies, tickets, users, administration) and is responsible for managing local state, calling APIs, and handling business logic on the client side. Hooks are designed with single-responsibility in mind – each function performs a specific task and avoids triggering UI side effects such as navigation or notifications. This separation ensures hooks are easy to reuse across components, simple to unit test, and flexible to extend as the system evolves. Additionally, user-facing and admin-facing hooks are clearly separated to maintain code clarity, enforce permission boundaries, and support scalable front-end architecture.



4.2.1 Authentication logic

AuthenticationLogic
+ useLogin() + useStaffLogin() + useRegister() + useActivateAccount() + useLogout() + useAuthInterceptor() + useForgotPassword() + useStaffForgotPassword() + useChangePassword() + useResetPassword()

These hooks handle all user authentication logic on the front-end, including: login, registration, logout, password reset, token-based user info retrieval, and login status checking.

Functions:

- useLogin():** Sends customer’s login credentials to the server. If valid, stores the token and user info into state. Staff’s account can not use this hook.
- useStaffLogin():** Sends staff’s login credentials to the server. If valid, stores the token and user info into state. Staff’s account can not use this hook.
- useRegister():** Sends user registration data (name, email, password...) to the server to create a new account.
- useActivateAccount():** A hook for a user to activate their account using a token after registration.
- useLogout():** Logs out the user: clears token, resets state, and optionally clears localStorage.
- useAuthInterceptor():** Automatically handles session expiry (401/403 errors) by logging the user out.

- useForgotPassword():** Sends a password reset request to the customer’s email. The server will send a reset link.
- useStaffForgotPassword():** Sends a password reset request to the staff’s email. The server will send a reset link.
- useChangePassword():** Changes password for the logged-in user, using the current password for verification
- useResetPassword():** Completes the password reset process using the token from the reset email and sets a new password for the account.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.2.2 User logic

UserLogic
+ useFetchProfile() + useUpdateProfile() + useGetWatchHistory() + useGetWishlist() + useAddToWishlist() + useRemoveFromWishlist() + useRateMovie() + useGetMyRatings() + useMessageChatBot()

These hooks manage user profile data and user-specific interactions such as watch history, wishlist, and movie ratings.

Functions:

useFetchProfile(): Retrieves the current user's profile from the server.

useUpdateProfile(): Sends updated profile data to the server and updates the global user state upon success.

useGetWatchHistory(): Retrieves the user's past movie viewing history.

useGetWishlist(): Retrieves the list of movies the user has added to their wishlist.

useAddToWishlist(): Adds a movie to the user's wishlist.

useRemoveFromWishlist(): Removes a movie from the wishlist.

useRateMovie(): Submits a star rating for a specific movie.

useGetMyRatings(): Retrieves the rating the user has given to a specific movie.

useMessageChatBot(): Sends a message to the AI assistant and returns the generated response for the user.

4.2.3 Movie logic

MovieLogic
+ useSetBranch() + useFetchNowShowing() + useFetchComingSoon() + useGetMovieDetail() + useGetShowtimes() + useSearchMovies() + useFilterByBranch()

These hooks handle movie-related logic such as retrieving movie lists, searching, filtering, and getting showtimes based on selected branches.

Functions:

useSetBranch(): Manages the active branch context. Returns functions to set the branch and get the current branch.

useFetchNowShowing(): Fetches the list of movies currently showing.

useFetchComingSoon(): Fetches the list of upcoming movies.

useGetMovieDetail(): Retrieves detailed information about a specific movie.

useGetShowtimes(): Fetches the showtimes for a specific movie at the current branch.

useSearchMovies(): Fetches a list of movies from the API that match a given search keyword.

useFilterByBranch(): Filters the movie list based on a specific branch.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.2.4 Branch logic

BranchLogic
+ useFetchBranches() + useGetBranchById() + useSetCurrentBranch() + useGetScreens() + useUpdateScreen() + useRemoveScreen() + useGetSchedules() + useUpdateSchedule() + useRemoveSchedule() + useGetSnacks() + useUpdateSnack() + useRemoveSnack()

These hooks handle all branch-level data operations such as fetching screens, schedules, snacks, and managing movie/staff within a specific branch.

Functions:

useFetchBranches(): Retrieves the list of all branches.

useGetBranchById(): Retrieves detailed information for a single, specified branch.

useSetCurrentBranch(): Manages the staff's currently selected branch, persisting the value to localStorage and providing a function to clear it.

useGetScreens(): Retrieves screens for the currently selected branch.

useUpdateScreen(): Updates screen details such as name, type, or capacity.

useRemoveScreen(): Deletes a screen from the selected branch.

useGetSchedules(): Retrieves scheduled movies for the current branch on a given date.

useUpdateSchedule(): Updates an existing schedule (if an ID is provided) or creates a new one for the branch.

useRemoveSchedule(): Removes a scheduled showtime from the branch.

useGetSnacks(): Fetches the list of snacks available at the selected branch.

useUpdateSnack(): Updates snack information such as name, price, or availability.

useRemoveSnack(): Deletes a snack item from the branch's menu.

4.2.5 Ticket logic

TicketLogic
+ useFetchAvailableSeats() + useGetSeatsBySchedule() + useApplyPromotion() + useCreateTicket() + useStartHoldSession() + useClearSession() + useCheckin() + useGetTicketDetailsByCode() + useGetSnacksByBranch()

These hooks manage the entire ticket booking logic on the front-end, including seat selection, snack selection, promotions, and ticket creation.

Functions:

useFetchAvailableSeats(): Retrieves the available seats for the selected schedule.

useGetSeatsBySchedule():

useApplyPromotion(): Applies a promotion code to the current booking.

useCreateTicket(): Sends the booking information to the server to create a ticket.

useStartHoldSession(): Initiates a session to temporarily hold selected seats.

useClearSession(): Clears all selected seats and snacks.

useCheckin(): Marks a ticket as checked in at the cinema.

useGetTicketDetailsByCode()

useGetSnacksByBranch()

4.2.6 Report logic

ReportLogic
+ useGetRevenueReport() + useGetAvailableBranches() + useSetBranch() + useSetDateRange() + useSetMovie()

These hooks handle fetching and filtering of revenue and performance reports by branch, movie, or date, with role-based access control.

Functions:

useGetRevenueReport(): Retrieves report data based on selected filters.

useGetAvailableBranches(): Returns the list of branches available for reporting.

useSetBranch(): Sets the reporting context to a selected branch.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

`useSetDateRange()`: Sets the date range for report filtering.
`useSetMovie()`: Sets the selected movie for filtering report results.

4.2.7 Administration logic

AdministrationLogic
+ <code>useGetAccounts()</code> + <code>useAddAccount()</code> + <code>useUpdateAccount()</code> + <code>useUpdateUserPermission()</code> + <code>useRemoveAccount()</code> + <code>useGetPromotions()</code> + <code>useAddPromotion()</code> + <code>useUpdatePromotion()</code> + <code>useRemovePromotion()</code> + <code>useGetMovies()</code> + <code>useAddMovie()</code> + <code>useUpdateMovie()</code> + <code>useRemoveMovie()</code> + <code>useAddBranch()</code> + <code>useUpdateBranch()</code> + <code>useRemoveBranch()</code>

These hooks handle system-wide administration tasks such as managing accounts, branches, movies, and promotions.

Functions:

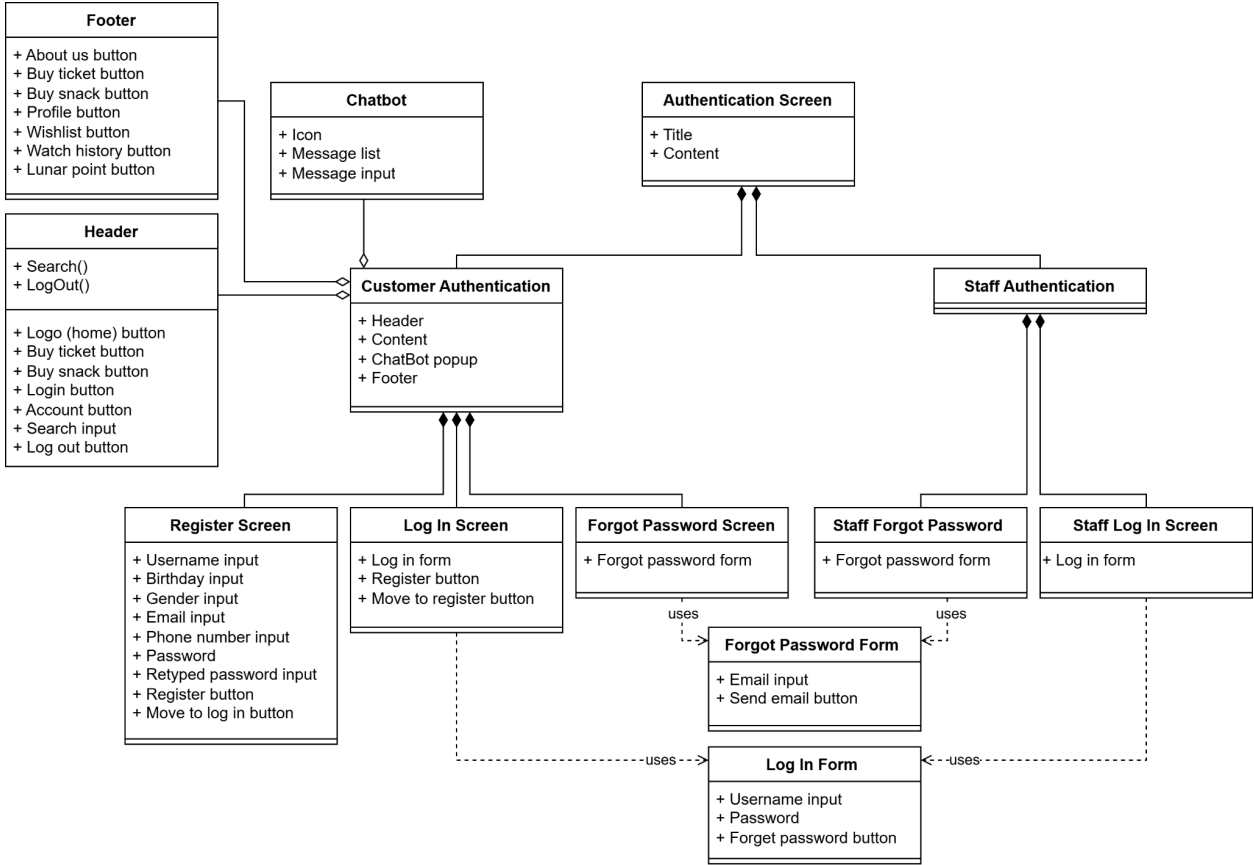
`useGetAccounts()`: Retrieves the list of user accounts available in the system.
`useAddAccount()`: Adds a new user account with specified roles and credentials.
`useUpdateAccount()`: Updates details of an existing user account.
`useUpdateUserPermission()`: Updates roles for an existing user account.
`useRemoveAccount()`: Deletes a user account from the system.
`useGetPromotions()`: Retrieves the list of all promotions in the system.
`useAddPromotion()`: Adds a new promotion to be applied to tickets or snacks.
`useUpdatePromotion()`: Updates details of an existing promotion.
`useRemovePromotion()`: Deletes a promotion from the system.
`useGetMovies()`: Retrieves the list of movies available for administration.
`useAddMovie()`: Adds a new movie to the system with full metadata.
`useUpdateMovie()`: Updates the details of an existing movie.
`useRemoveMovie()`: Removes a movie from the system.
`useAddBranch()`: Adds a new cinema branch with its information.
`useUpdateBranch()`: Updates details of an existing branch.
`useRemoveBranch()`: Removes a branch from the system.

4.3 Component: User Interface (UI)

The UI component layer of Lumiere Cinema is structured around five main layouts: Authentication, Main, Ticket Booking, User Profile, and Admin Dashboard. Each layout encapsulates related screens and interactions, ensuring clear separation of concerns and consistent user experience across different roles. Components are designed to be modular and reusable, with shared elements like Header, Footer, and Chatbot integrated into the Main layout. Navigation flows logically between components – for example, users can browse movies, proceed to book tickets, and later access their booking history through the profile section. This architecture supports both customer and staff, providing a flexible and scalable interface that aligns with the underlying hook and service layers.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

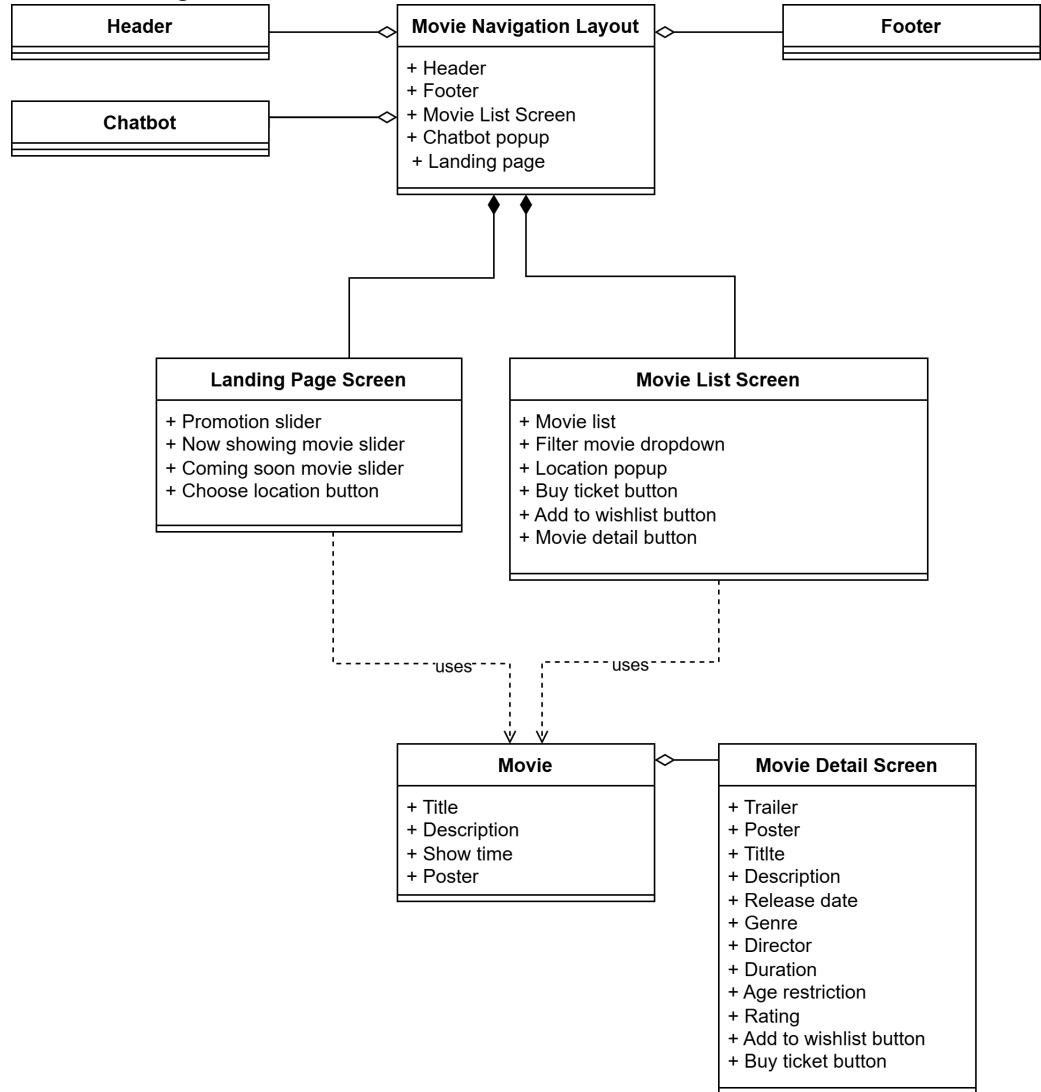
4.3.1 Authentication layout



This package manages user authentication, including registration, login, and password recovery screens. It distinguishes between staff and customer login flows: staff have access only to login and password reset, with a minimal UI optimized for fast interaction, while customers can register, log in, and reset passwords, and also navigate customer-specific screens via integrated header and footer components. The package ensures each user is properly authenticated and authorized to access the screens corresponding to their role.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.3.2 Movie navigation



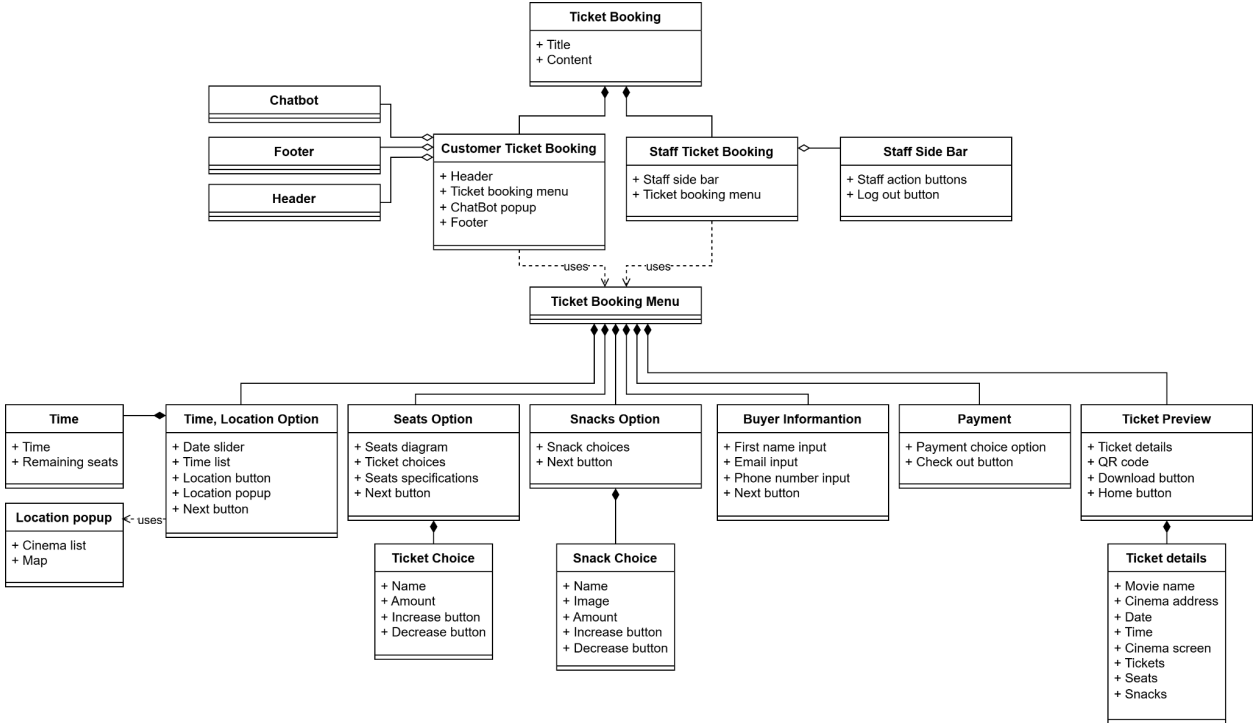
This diagram illustrates the user interface layout for the Movie Navigation Component, which supports user interaction with movie-related content and ticket purchasing features. The main structure is organized around the Movie Navigation Layout, which integrates reusable components such as Header, Footer, and Chatbot, along with dedicated movie navigation screens.

From the Movie Navigation Layout, users can access the following sub-screens:

- **Landing Page Screen:** Serves as the homepage for the movie portal. It includes a promotion slider, sections for now showing and coming soon movies, and allows users to select their preferred cinema location.
- **Movie List Screen:** Displays a scrollable list of movies. Users can filter movies, choose a cinema location, view movie details, add movies to their wishlist, or proceed to ticket booking.
- **Movie Detail Screen:** Shows comprehensive information about a selected movie, including trailer, poster, description, release date, genre, director, duration, rating, and age restriction. From here, users can also add the movie to their wishlist or buy a ticket.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

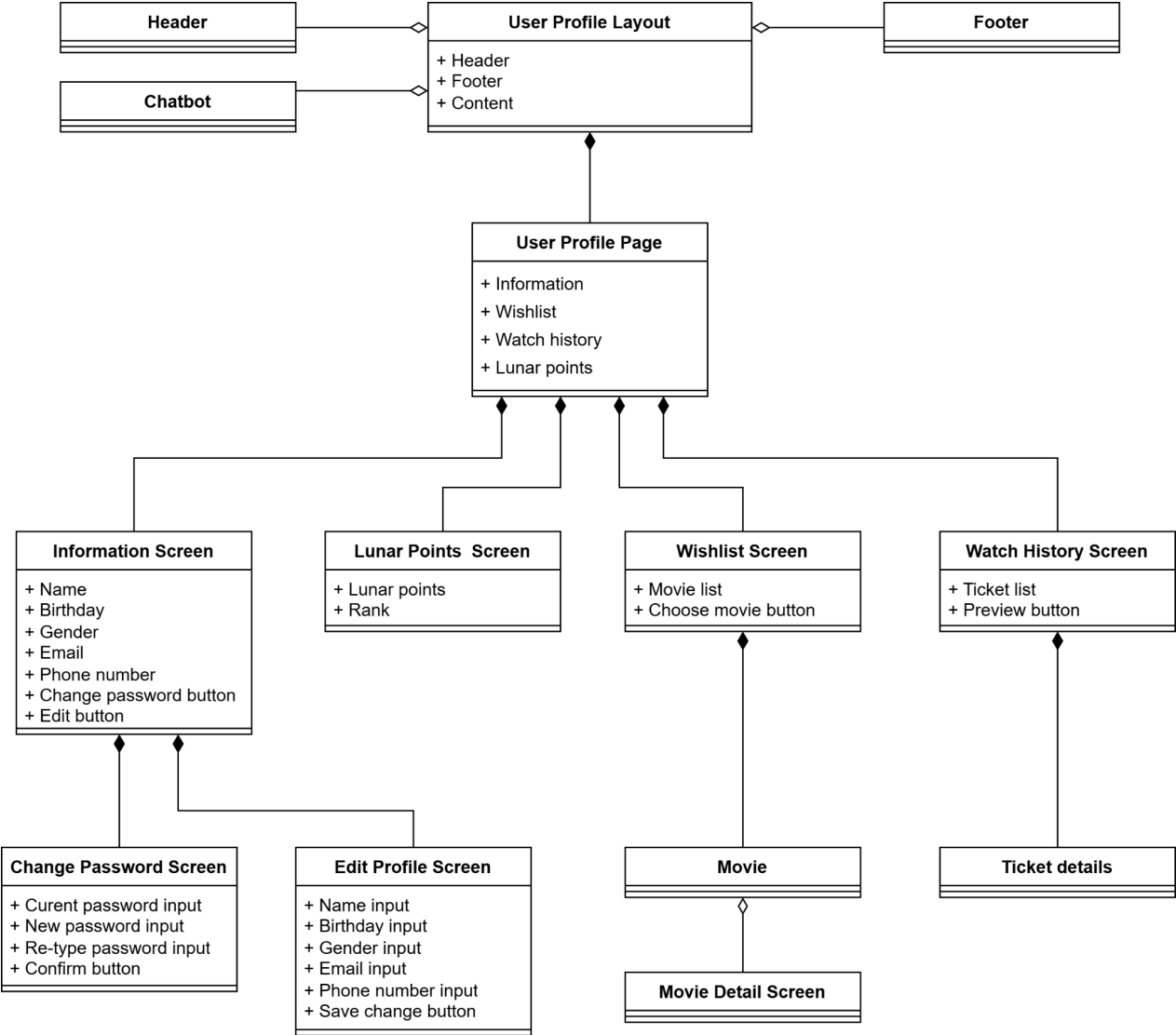
4.3.3 Ticket booking flow



This package manages the ticket booking process for both movie and snack tickets. The customer layout includes reusable components such as a header, footer, and chatbot popup, while the staff layout is simpler, featuring a sidebar for easy access to staff-specific functions. Despite the differences in layout, both customer and staff interfaces utilize the core component of this package: the Ticket Booking Menu. This menu guides users through a step-by-step process, including selecting time and location, choosing seats and snacks, entering buyer information, selecting a payment method, and finally, previewing the completed ticket.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.3.4 User profile



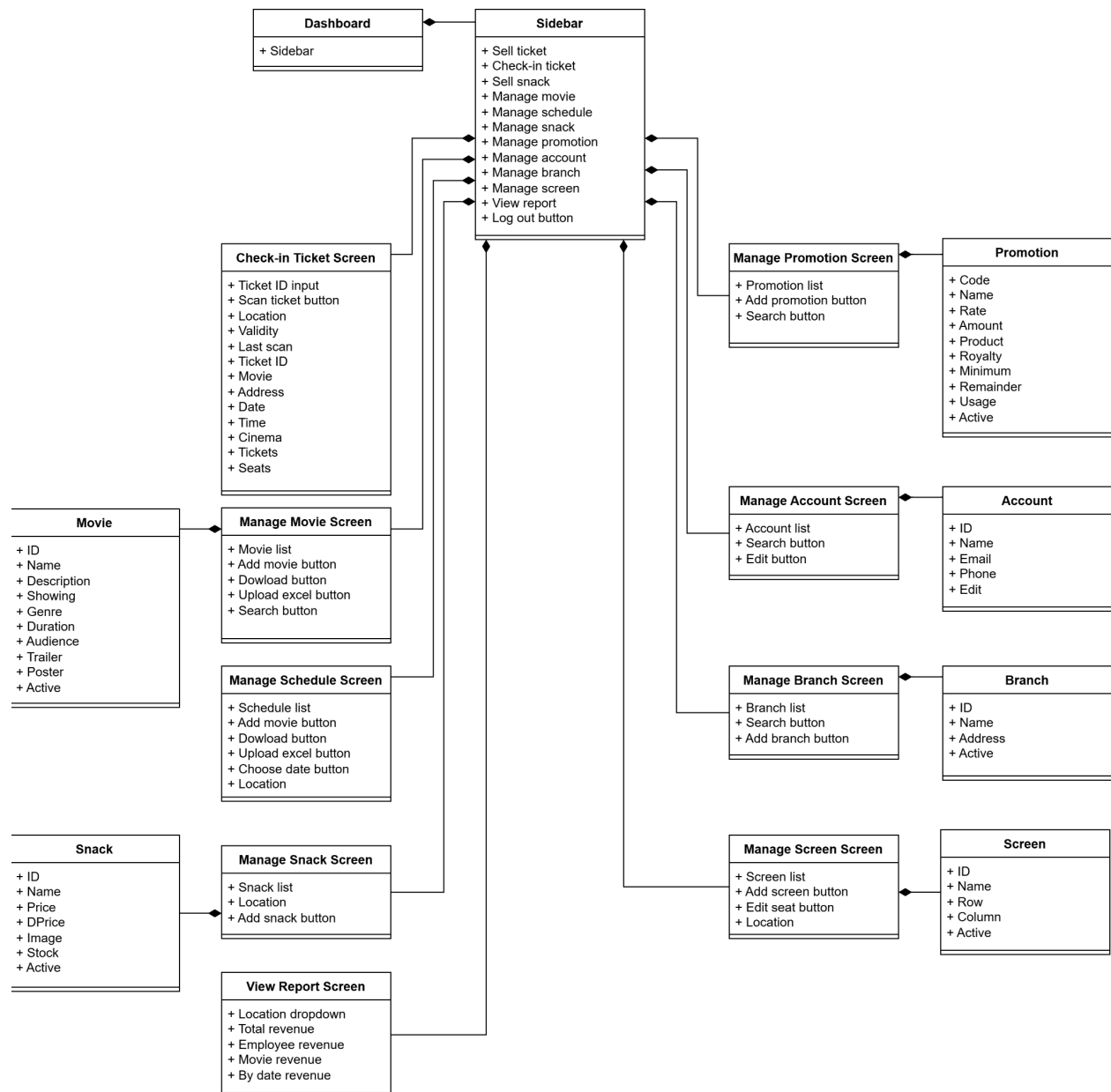
This diagram illustrates the user interface layout for the user profile section. The main component is `UserProfileLayout`, which includes reusable parts like `Header`, `Footer`, and the central content area: `User Profile Page`.

From the `User Profile Page`, users can access the following sub-screens:

- `Information Screen`: Displays personal details such as name, birthday, gender, email, and phone number. It also provides access to two subscreens:
 - + `Edit Profile Screen`: Allows users to update their personal information.
 - + `Change Password Screen`: Allows users to securely change their password.
- `Wishlist Screen`: Shows the user's favorite movies and allows selecting a movie.
- `Watch History Screen`: Lists previously viewed tickets with preview options.
- `Lunar Points Screen`: Displays the user's loyalty points and ranking.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.3.5 Dashboard layout



This diagram illustrates the user interface layout for the Dashboard Component of the cinema management system. The central component is the Dashboard, which includes a reusable Sidebar for navigation across various management and operational screens.

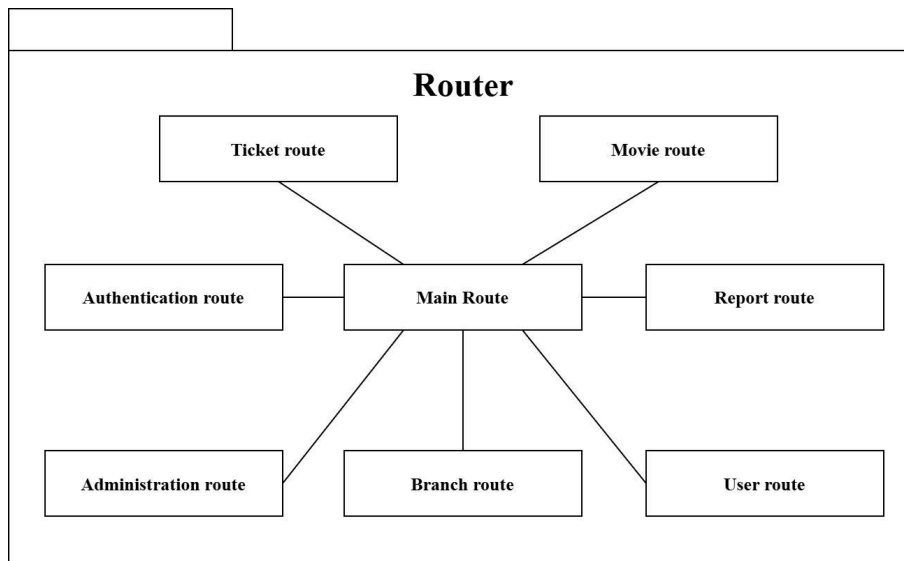
From the Sidebar, users can access the following sub-screens:

- Check-in Ticket Screen: Allows staff to scan ticket IDs, verify validity, and view details such as movie, date, time, and cinema location.
- Manage Movie Screen: Displays the list of available movies and provides functions to add, search, download, or upload movie data.
- Manage Schedule Screen: Enables administrators to manage screening schedules, with options to add movies, filter by location, and handle Excel uploads/downloads.
- Manage Snack Screen: Shows the snack list and allows filtering by cinema branch.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

- Manage Promotion Screen: Manages promotion codes and their attributes such as rate, product type, royalty, and usage statistics.
- Manage Account Screen: Displays user account information with features to search and edit account details.
- Manage Branch Screen: Allows administrators to manage cinema branches including their names, addresses, and statuses.
- Manage Screen Screen: Provides functionality to manage cinema screens (halls), including editing screen layout and filtering by branch.
- View Report Screen: Displays financial and operational statistics such as total revenue, revenue by employee, movie revenue, and filters for location and date.

4.4 Component: Routers



The *main route* is the designated place to apply middleware functions (CORS, Body Parsing,..) that are executed for all incoming requests across the system. After processing global middleware, the Main Route delegates requests to their corresponding specialized sub-routers based on their Base URL. This ensures that each type of resource (authentication, user, movie, branch, ticket, report, administration) is handled by its dedicated route module, promoting a well-structured and modular back-end architecture

For example: The Lumiere Cinema login begins when a user interacts with the UI Component, triggering its `login()` function. This calls the `useAuthentication` Hook, which then sends an HTTP POST request via the `AuthenticationService` to the back-end's `/api/auth/login` endpoint. Upon arrival, the Router recognizes `/api/auth/login` as an endpoint and directs it to the `Authentication Controller`.

4.4.1 Authentication routes

Base URL: `/api/auth`

- POST Routes:

- + `/login`: Authenticates customer's account credentials (email and password). If valid, the server generates an Access Token and Refresh Token, then returns them along with user info to the client for session management.
- + `/staff/login`: Authenticates staff's account credentials (email and password). If valid, the server generates an Access Token and Refresh Token, then returns them along with user info to the client for session management.
- + `/register`: Registers a new user account. The server securely hashes the password and stores the user's information in the database.
- + `/activate/:token`: This route is used to activate a new user's account after they have registered, likely by clicking a link sent to their email.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

- + **/logout**: Logs the user out of the system. The server invalidates the current refresh token.
 - Requires Authentication: Customer, Cashier, Checkin Counter, Branch Manager, Administrator.
- + **/forgot-password**: Initiates a password reset process for customers. The server generates a secure reset token and sends a password reset link to the user's email address.
- + **/staff/forgot-password**: Initiates a password reset process for staff. The server generates a secure reset token and sends a password reset link to the user's email address.
- + **/reset-password**: Finalize the password change using a token sent to the user's email. With the provided reset token, users can create a new password for their accounts.
- + **/change-password**: Allows the currently authenticated user to change their password. The server verifies the current password before updating to the new one to ensure security.
 - Requires Authentication: Customer, Cashier, Checkin Counter, Branch Manager, Administrator.

4.4.2 User routes

Base URL: **/api/users**

- **GET Routes:**
 - + **/me**: Get the profile details of the currently authenticated user.
 - Requires Authentication: Customer
 - + **/tickets**: Get a list of movie tickets booked by the currently authenticated user.
 - Requires Authentication: Customer
 - + **/wishlist**: Retrieves the list of movies the authenticated user has added to their wishlist.
 - Requires Authentication: Customer
 - + **/watch-history**: Retrieves the user's watch history.
 - Requires Authentication: Customer
- **POST Routes:**
 - + **/wishlist/:movieId**: Adds a specified movie to the authenticated user's wishlist.
 - Requires Authentication: Customer
 - + **/rate/:movieId**: Submits a new rating for a movie that the authenticated user has previously watched and is eligible to rate.
 - Requires Authentication: Customer
- **PUT Routes:**
 - + **/me**: Fully updates the profile details of the currently authenticated user. All editable fields must be provided.
 - Requires Authentication: Customer
- **PATCH Routes:**
 - + **/me**: Performs a partial update on the profile details of the currently authenticated user. Only the fields provided in the request body will be modified.
 - Requires Authentication: Customer
- **DELETE Routes:**
 - + **/wishlist/:movieId**: Removes a specific movie from the authenticated user's wishlist.
 - Requires Authentication: Customer
 - + **/watch-history/:ticketId**: Removes a specific ticket from the user's watch history.
 - Requires Authentication: Administrator

4.4.3 Movie route

Base URL: **/api/movies**

- **GET Routes:**
 - + **/now-showings**: Retrieves a list of movies that are currently being shown.
 - + **/upcoming**: Retrieves a list of upcoming movies.
 - + **/search**: Searches for movies based on keywords or criteria like genre.
 - + **/:movieId**: Retrieves detailed information for a specific movie, including synopsis, genre, trailer, and screening schedules.
 - + **/:movieId/showscreen**: Retrieves all screenings schedules for a given movie.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

- + **/all**: Retrieves a comprehensive list of all movies in the system for management purposes.
 - Requires Authentication: Administrator
- + **/:movieId/get-ratings**: Retrieves rating of a specific movie.
- **POST Routes:**
 - + **/**: Adds a new movie to the system, including its information (title, genre, duration, etc.).
 - Requires Authentication: Administrator
- **PUT Routes:**
 - + **/:movieId**: Fully updates all details of an existing movie.
 - Requires Authentication: Administrator
- **PATCH Routes:**
 - + **/:movieId**: Partially updates specific fields of an existing movie's information.
 - Requires Authentication: Administrator
- **DELETE Routes:**
 - + **/:movieId**: Removes a movie from the system or deactivates it from public display.
 - Requires Authentication: Administrator

4.4.4 Branch route

Base URL: **/api/branches**

- **GET Routes:**
 - + **/available**: Retrieves a list of available branches.
 - + **/:branchId/screens**: Retrieves a list of screens/halls available at a specific branch.
 - Requires Authentication: Branch Manager
 - + **/:branchId/snacks**: Retrieves the list of available snack combos and individual snack items for a specific cinema branch
 - Requires Authentication: Branch Manager, Customer, Cashier, Administrator
 - + **/:branchId/schedules**: Retrieves all movie screening schedules for a specific branch, optionally filtered by date.
 - Requires Authentication: Branch Manager
 - + **/:branchId/screens**: Retrieves a list of all screens within a specific branch.
 - Requires Authentication: branchmanager
 - + **/:branchId/screens/:screenId**: Retrieves detailed information for a specific screen.
 - Requires Authentication: branchmanager
- **POST Routes:**
 - + **/:branchId/screens**: Creates a new cinema screen within a specific branch..
 - Requires Authentication: Branch Manager
 - + **/:branchId/snacks**: Adds a new snack item to a specific branch's snack list.
 - Requires Authentication: Branch Manager
 - + **/:branchId/schedules**: Sets a new movie screening at a specific screen and time for a branch.
 - Requires Authentication: Branch Manager
- **PUT Routes:**
 - + **/:branchId/screens/:screenId**: Fully updates details of an existing screen within a specific branch.
 - Requires Authentication: Branch Manager
 - + **/:branchId/snacks/:snackId**: Fully updates details of an existing snack item for a specific branch..
 - Requires Authentication: Branch Manager
 - + **/:branchId/schedule-detail**: Fully updates the movie screening at a specific screen and time for a branch.
 - Requires Authentication: Branch Manager
- **PATCH Routes:**
 - + **/:branchId/screens/:screenId**: Partially updates details of an existing screen within a specific branch.
 - Requires Authentication: Branch Manager

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

- + `/:branchId/snacks/:snackId`: Partially updates details of an existing snack item for a specific branch.
 - Requires Authentication: Branch Manager
- + `/:branchId/schedule-detail`: Partially modifies details of an existing movie schedule for a specific branch.
 - Requires Authentication: Branch Manager
- **DELETE Routes:**
 - + `/:branchId/screens/:screenId`: Removes a specific screen from a specific branch.
 - Requires Authentication: Branch Manager
 - + `/:branchId/snacks/:snackId`: Removes a snack item from a specific branch's list.
 - Requires Authentication: Branch Manager
 - + `/:branchId/schedule-detail`: Cancels an existing movie schedule for a specific branch.
 - Requires Authentication: Branch Manager

4.4.5 Ticket route

Base URL: `/api/tickets`

This API section manages all operations related to movie tickets and snack purchases within the Lumiere Cinema system. It provides functionalities for users to view their purchased tickets and for authorized personnel (Customers or Cashiers for purchase, Admins for full management) to interact with ticket data.

Ticket Routes: `/api/tickets/movies`

This subset of routes specifically handles movie ticket functionalities, linking users to their booked seats for specific movie schedules.

- **GET Routes:**
 - + `/:branchId/schedule`: Retrieves schedules by branch.
 - + `/screen/:scheduleId`: Retrieves the seat map and status for a specific schedule.
 - + `/create`: Creates a unified ticket (movie and/or snack).
 - + `/movie/admin/all`: Retrieves all movie tickets.
 - Requires Authentication: administrator
 - + `/movie/admin/:ticketCode`: Retrieves a movie ticket by its code.
 - Requires Authentication: administrator
 - + `/snack/admin/:ticketCode`: Retrieves a specific snack ticket by its code.
 - Requires Authentication: administrator
 - + `/create`: Creates a unified ticket, which can include a movie ticket, snack ticket, or both in a single transaction.
 - Requires Authentication: administrator, cashier
- **POST Routes:**
 - + `/movie/hold`: Temporarily holds seats for a schedule.
 - + `/movie/hold/cleanup`: Cleans up expired seat holds
 - Requires Authentication: administrator
 - + `/snack/create`: Creates a standalone snack ticket
- **PATCH Routes:**
 - + `/movie/hold/:holdId`: Manages a seat hold (release or extend).
 - + `/movie/admin/:ticketCode`: Updates a movie ticket.
 - Requires Authentication: administrator
- **DELETE Routes:**
 - + `/movie/hold/bulk`: Bulk releases seat holds for a user/session.
 - + `/snack/admin/:ticketCode`: Cancels a snack ticket.
 - Requires Authentication: administrator

4.4.6 Chatbot route

Base URL: `/api/chatbot`

- **POST Routes:**

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

- + **/query**: Send user's query

4.4.7 QR route

Base URL: **/api/qr**

- GET Routes:

- + **/**: Retrieves QR code as PNG buffer with custom size.

4.4.8 Administration route

Base URL: **/api/admin**

- GET Routes:

- + **/users/**: Retrieves a comprehensive list of all user accounts within the system, along with key details and management options for each account.
 - Requires Authentication: Administrator
- + **/users/:userId**: Retrieves detailed profile information for a specific user account.
 - Requires Authentication: Administrator
- + **/promotions/all**: Retrieves a comprehensive list of all promotional campaigns (including inactive or expired ones) for management purposes.
 - Requires Authentication: Administrator
- + **/promotions/:promoCode**: Retrieves a promotional campaign.
 - Requires Authentication: Administrator

- POST Routes:

- + **/user**: Adds a new user account.
 - Requires Authentication: Administrator
- + **/promotion**: Adds a new promotional campaign to the system.
 - Requires Authentication: Administrator
- + **/branch**: Adds a new branch.
 - Requires Authentication: Administrator

- PUT Routes:

- + **/users/:userId**: Fully updates the profile details of any user account.
 - Requires Authentication: Administrator
- + **/promotions/:promoCode**: Fully updates all details of an existing promotional campaign.
 - Requires Authentication: Administrator
- + **/branch/:branchId**: Fully updates the branch information.
 - Requires Authentication: Administrator

- PATCH Routes:

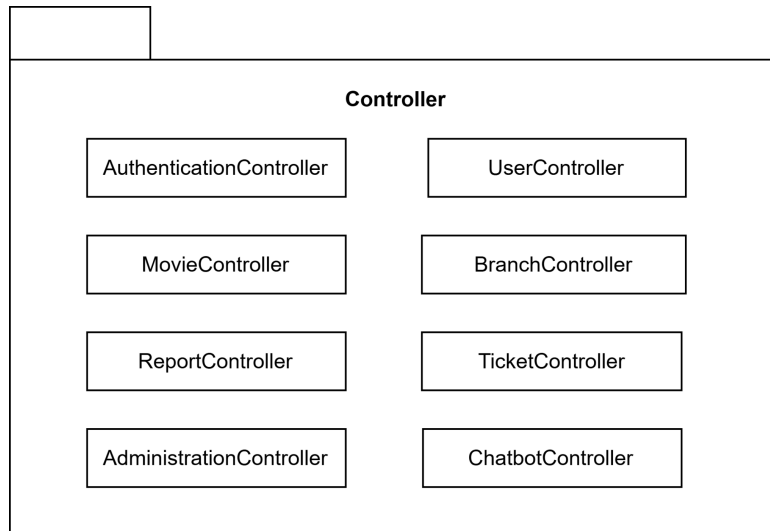
- + **users/:userId**: Partially updates the profile details of any user account.
 - Requires Authentication: Administrator
- + **/promotions/:promoCode**: Partially updates specific fields of an existing promotional campaign.
 - Requires Authentication: Administrator
- + **/users/:userId/role**: Manages the roles assigned to a specific user account by adding or removing roles from a list of available options.
 - Requires Authentication: Administrator
- + **/branch/:branchId**: Partially updates the branch information.
 - Requires Authentication: Administrator

- DELETE Routes:

- + **/users/:userId**: Deletes a user account from the system.
 - Requires Authentication: Administrator
- + **/promotions/:promoCode**: Remove a promotional campaign.
 - Requires Authentication: Administrator
- + **/branch/:branchId**: Remove a branch.
 - Requires Authentication: Administrator

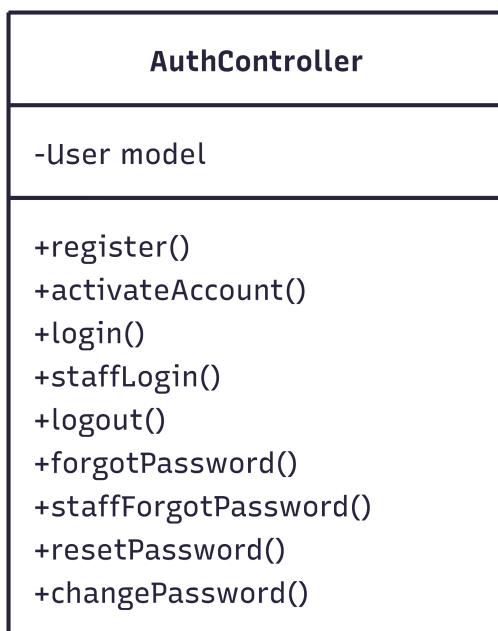
Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.5 Component: Controllers



These controllers are responsible for receiving requests from routes, parsing them, and processing them within the Model component. Controllers also format responses, ensuring data is returned consistently and handling HTTP status codes, thereby maintaining a clear interface between clients and the back-end.

4.5.1 Authentication controller



register() : Creates a new user account, hashes the password, and sends an activation email.

activateAccount() : Validates an account activation token from an email link and activates the user's account.

login() : Logs in a customer by verifying their credentials and returns a JWT.

staffLogin() : Logs in a staff member by verifying their credentials and role, then returns a JWT.

logout() : Returns a successful logout message.

forgotPassword() : Sends a password reset link to a customer's email

staffForgotPassword() : Sends a password reset link to a staff member's email.

resetPassword() : Resets a user's password using a valid token from an email link.

changePassword() : Allows an authenticated user to change their password by providing their current password.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.5.2 User controller

UserController
-User model -Movie model -Ticket model -MovieRating model -SnackTicket model
+getProfile() +updateProfile() +rateMovie() +getRatingMovie() +addToWishlist() +removeFromWishlist() +getWishlist() +getWatchHistory() +removeFromWatchHistory() +getUserTickets()

getProfile() : Retrieves the profile of the currently authenticated user

updateProfile() : Partially updates the authenticated user's profile with allowed fields (name, phone, birthday, gender)

rateMovie() : Creates a new rating or updates an existing one for a specific movie by the authenticated user

getRatingMovie() : Retrieves the authenticated user's specific rating for a single movie

addToWishlist() : Adds a movie to the authenticated user's wishlist

removeFromWishlist() : Removes a movie from the authenticated user's wishlist

getWishlist() : Retrieves the authenticated user's wishlist, populated with movie details

getWatchHistory() : Retrieves the authenticated user's watch history, populated with ticket details

removeFromWatchHistory() : Removes a single ticket from the authenticated user's watch history

getUserTickets() : Retrieves a paginated list of the authenticated user's tickets from the last 30 days, with optional status filtering

4.5.3 Movie controller

MovieController
-Movie model -Schedule model -MovieRating model
+getNowShowingMovies() +getUpcomingMovies() +getMovieDetails() +searchMovies() +getAllMovies() +addMovie() +updateMovie() +deleteMovie() +getMovieShowtimes() +getMovieRatingSummary()

getNowShowingMovies() : Retrieves a list of movies currently showing, determined by a release date that is on or before the current date

getUpcomingMovies() : Retrieves a list of movies that are scheduled for future release, determined by a release date after the current date

getMovieDetails() : Retrieves comprehensive details for a specific movie using its ID, sourced from a Redis cache or the database

searchMovies() : Enables users to locate relevant movies by specifying search criteria, utilizing MongoDB Atlas Search for fuzzy matching

getAllMovies() : Retrieves a comprehensive list of all movies in the system for management purposes

addMovie() : Adds a new movie to the system after checking for existing titles and clears relevant caches

updateMovie() : Updates an existing movie's details and clears related caches

deleteMovie() : Soft-deletes a movie by setting its isHidden flag to true, and clears related caches. If the movie has active schedules, it only returns a message without hiding the movie

getMovieShowtimes() : Retrieves specific screening schedules for a given movie, with an option to filter by date

getMovieRatingSummary() : Retrieves all ratings for a specific movie

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.5.4 Branch controller

createSnack() : Adds a new snack item to a specific branch's snack list.

editSnack() : Modifies details of an existing snack item.

deleteSnack() : Removes a snack item from a branch's list (with a soft-delete fallback).

getSnackList() : Retrieves the list of available snack items for a specific branch.

getAvailableBranches() : Retrieves a list of all active branches, including a count of currently showing movies for each.

getBranchById() : Retrieves detailed information for a single branch, including lists of its screens and currently showing movies.

scheduleMovieScreening() : Sets a new movie screening at a specific screen and time, including conflict detection.

editMovieSchedule() : Modifies details of an existing, upcoming movie schedule that has no tickets sold.

deleteMovieSchedule() : Cancels an existing, upcoming movie schedule that has no tickets sold.

getMovieSchedules() : Retrieves a filterable, paginated list of all movie schedules for a specific branch.

getBranchScreens() : Retrieves a list of all screens for a specific branch.

createScreen() : Creates a new cinema screen within the manager's assigned branch.

getScreenById() : Retrieves detailed information for a single screen within the manager's assigned branch.

updateScreen() : Modifies details of an existing screen.

deleteScreen() : Removes a screen from a specific branch (performs a soft-delete if schedules exist).

BranchController
-Branch model -Snack model -Screen model -Schedule model -Movie model -Ticket model
+createSnack() +editSnack() +deleteSnack() +getSnackList() +getAvailableBranches() +getBranchById() +scheduleMovieScreening() +editMovieSchedule() +deleteMovieSchedule() +getMovieSchedules() +getBranchScreens() +createScreen() +getScreenById() +updateScreen() +deleteScreen()

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.5.5 Report controller

ReportController
-Ticket model -Branch model -Movie model -User model
+getTotalRevenue() +getBranches() +getBranch() +getRevenueSummary()

getTotalRevenue(): Retrieves comprehensive revenue data, including daily totals and a summary, based on a specified date range and optional branch filter.

getBranches(): Retrieves a list of all active branch names, utilizing a 24-hour cache for performance.

getBranch(): Retrieves the assigned branch for the currently authenticated Branch Manager.

getRevenueSummary(): Retrieves a detailed revenue summary with multiple breakdowns (by date, by employee, by movie) for a given date range, automatically filtered for the logged-in user's branch if they are not an administrator.

4.5.6 Ticket controller

TicketController
-Ticket model -SnackTicket model -Schedule model -SeatHold model -Snack model -User model -Branch model -Promotion model
+getSchedulesByBranch() +getSeatMapBySchedule() +createTicket() +createSnackTicket() +holdSeats() +manageSeatHold() +releaseBulkHolds() +cleanupExpiredHolds() +getSnacksByBranch() +reserveSnacks() +getTicketByCode() +getAllTickets() +updateTicket() +deleteTicket()

getSchedulesByBranch(): Retrieves detailed movie schedules for a specific branch, including real-time seat availability.

getSeatMapBySchedule(): Generates a detailed seat map for a specific schedule, showing the status (available, occupied, holding) and price of every seat.

createTicket(): Creates a final ticket for movies, snacks, or both, processing payment, applying promotions, updating seat/stock status, and awarding loyalty points within a single transaction.

createSnackTicket(): Creates and finalizes a standalone snack ticket, processing payment details and updating stock.

holdSeats(): Temporarily reserves specific seats for a schedule using a transaction to prevent race conditions.

manageSeatHold(): Manages an existing seat hold, allowing the owner to release or extend it.

releaseBulkHolds(): Releases all active seat holds for a specific user or session on a given schedule.

cleanupExpiredHolds(): (Admin) Cleans up and deletes all expired seat holds from the system.

getSnacksByBranch(): Retrieves the list of available snacks for a specific branch, including real-time stock information.

reserveSnacks(): Temporarily reserves snack items by checking and updating stock levels within a transaction.

getTicketByCode(): Retrieves a specific ticket by its unique code.

getAllTickets(): Retrieves a list of all tickets (currently implemented for snacks, with a placeholder for movies).

updateTicket(): Updates a ticket's details, such as its status (currently implemented for snacks).

deleteTicket(): Cancels a ticket and releases associated seats if it's a movie ticket.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.5.7 Administration controller

createUser () : Creates a new user account (customer or staff) with specified roles, bypassing email activation.

getAllProfiles () : Retrieves a list of all user profiles in the system.

getDetailedProfile () : Retrieves detailed profile information for a specific user.

updateUserDetails () : Partially updates the profile details (e.g., name, phone) of a specific user.

updateUserRoles () : Manages the roles assigned to a specific user account.

updateUserStatus () : Changes the locked/unlocked status of a user account.

deleteUser () : Deletes a user account from the system, only if they have no associated tickets or ratings.

getAllPromotions () : Retrieves a comprehensive list of all promotional campaigns.

getPromotionByCode () : Retrieves a single promotional campaign by its code.

createPromotion () : Creates a new promotional campaign.

updatePromotion () : Updates an existing promotional campaign.

deletePromotion () : Deletes a promotional campaign from the system.

createBranch () : Creates a new cinema branch in the system.

updateBranch () : Updates the details of an existing cinema branch.

deleteBranch () : Deletes a cinema branch from the system, only if it has no associated staff, screens, or tickets.

updateBranchStatus () : Activates or deactivates a specific cinema branch.

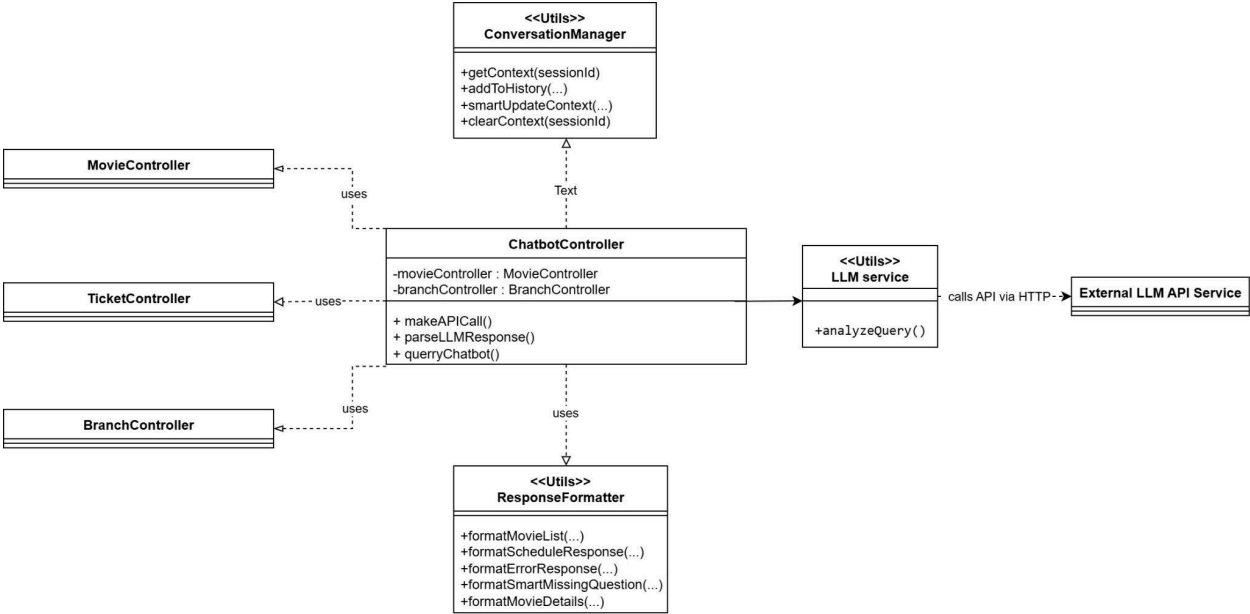
AdminController

-User model
-Promotion model
-Branch model
-Ticket model
-MovieRating model
-Screen model

+createUser()
+getAllProfiles()
+getDetailedProfile()
+updateUserDetails()
+updateUserRoles()
+updateUserStatus()
+deleteUser()
+getAllPromotions()
+getPromotionByCode()
+createPromotion()
+updatePromotion()
+deletePromotion()
+createBranch()
+updateBranch()
+deleteBranch()
+updateBranchStatus()

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.5.8 Chat-bot controller



Acts as an intermediary, enabling users to interact with an AI chatbot for movie-related inquiries using natural language. It interprets user queries, communicates with the Large Language Model (LLM) to understand intent and extract information, and then invokes relevant functions in other internal controllers to provide accurate responses.

Functions:

queryChatbot(): Processes natural language queries from the user, forwards them to the LLM service for analysis, and then orchestrates calls to appropriate functions in other controllers (like MovieController or BranchController) based on the LLM's understanding of the user's intent.

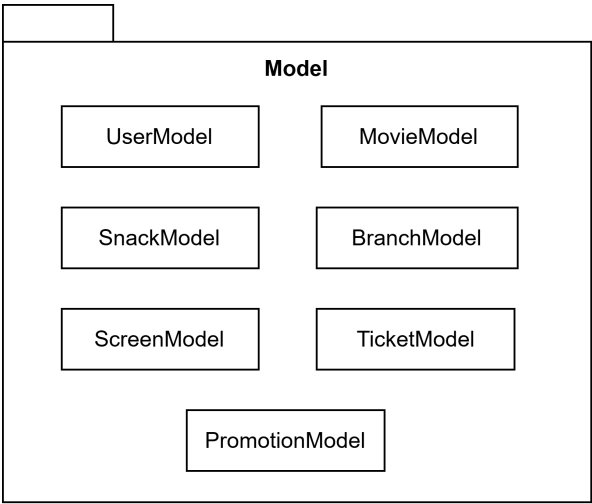
makeLLMApiCall(): Makes the actual HTTP API call to the external Large Language Model (LLM) service, sending the user's query and handling the communication protocol.

parseLLMResponse(): Parses and extracts structured information (like identified intent and relevant entities) from the raw response received from the LLM service.

4.6 Component: Model

The Model component encapsulates the application's domain knowledge, managing the data and defining the operations that manipulate this data. This component acts as the central repository for the application's state and behavior, ensuring data integrity and consistency across all functionalities.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

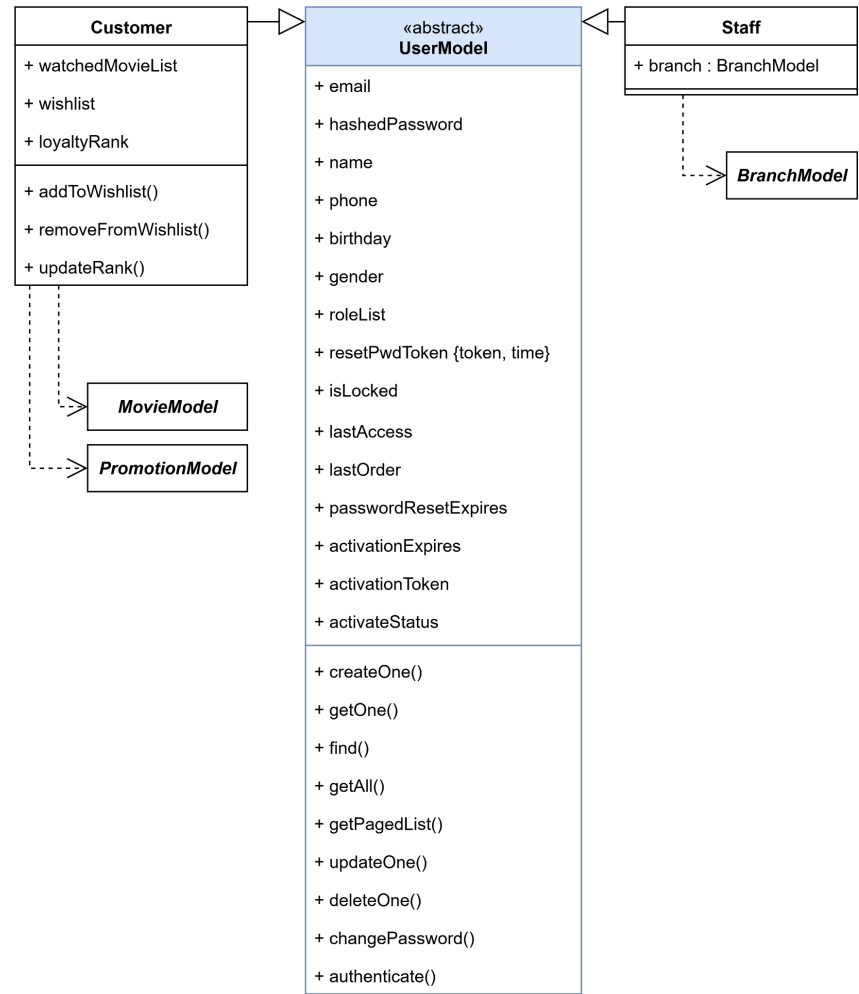


There are 7 main models, each model has its own purposes and features.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.6.1 UserModel

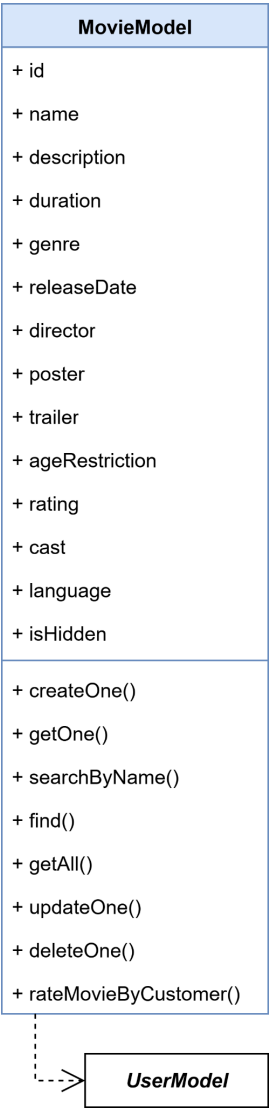
The UserModel is a critical part of the system's core, responsible for managing all user-related data and functionalities. It defines the structure of different user types and encapsulates the logic for user authentication, profile management, and loyalty program interactions. This model ensures a secure and personalized experience for both customers and internal staff.



Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.6.2 *MovieModel*

MovieModel serves as the central entity for managing all information related to movies available in the Lumiere Cinema system. It encapsulates the complete set of data necessary to describe a movie to users and for internal management.



Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.6.3 *SnackModel*

The *SnackModel* is designed to represent and manage the various food and beverage items offered for sale at Lumiere Cinema branches. It encapsulates the core details of each snack product, including its pricing, current availability (stock), and general information.



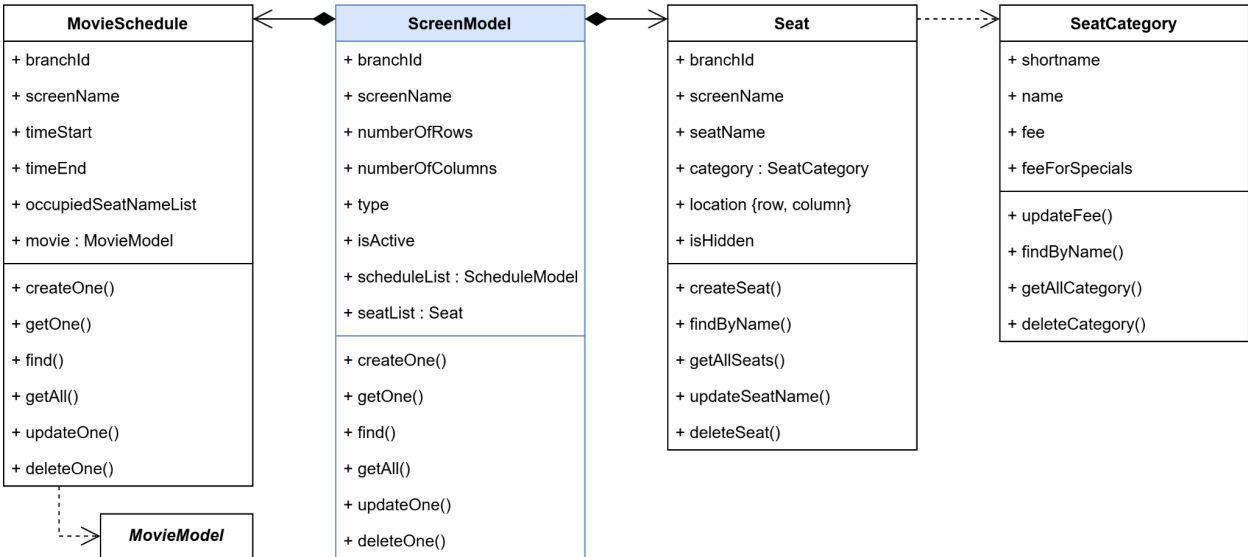
Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.6.4 BranchModel

The BranchModel serves as a central entity representing a physical cinema location of Lumiere Cinema. It encapsulates the core administrative and operational details specific to each branch, acting as a container for its screens and the snack products it offers.



4.6.5 ScreenModel



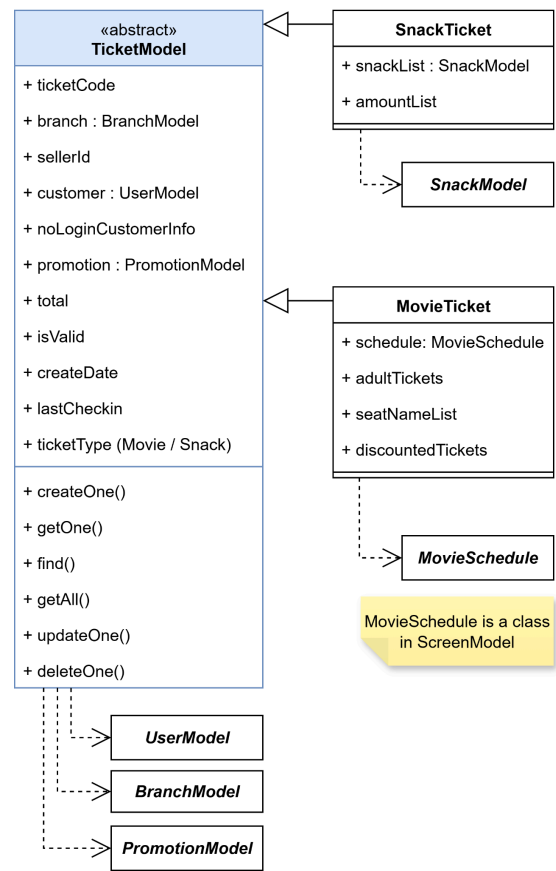
The ScreenModel class represents an individual movie screening room located within a cinema branch. It

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

encapsulates the physical and operational characteristics of each screen, serving as a container for its seating layout and a key component in scheduling movie showtimes.

4.6.6 TicketModel

The TicketModel class within the Model component is a core entity that represents a single purchased item in the Lumiere Cinema system. This can be a ticket for a specific movie seat at a particular showtime, or it can be a ticket for a snack item. It encapsulates the details of the transaction and the status of the purchased item.



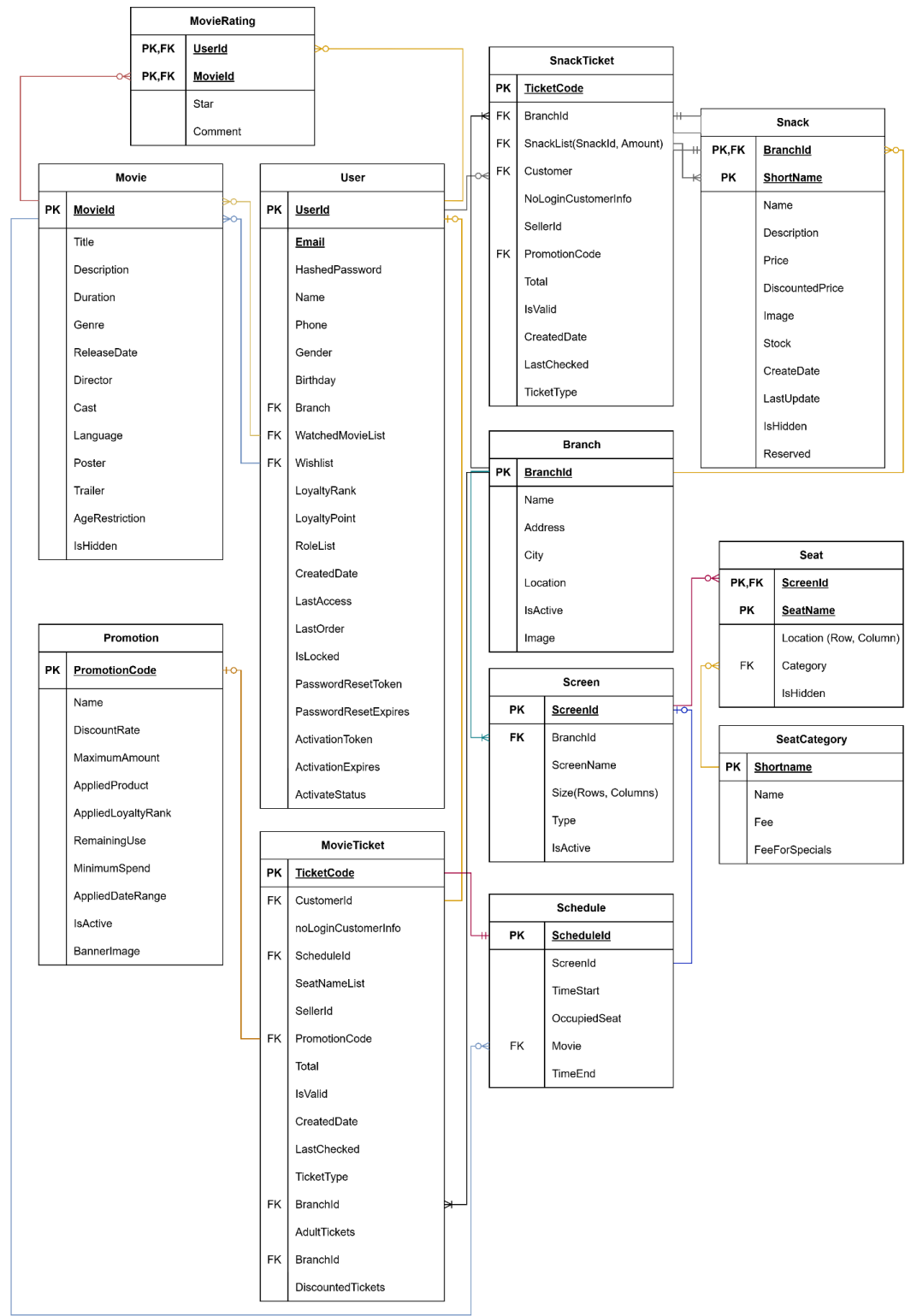
Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

4.6.7 PromotionModel

The PromotionModel class within the Model component is designed to manage all information regarding promotional offers and discount campaigns within the Lumiere Cinema system. It encapsulates the details defining various types of promotions and their applicability, playing a crucial role in the system's marketing and loyalty programs.

PromotionModel
+ promoCode
+ promoName
+ discountRate
+ maximumAmount
+ productRestriction
+ loyaltyRestriction
+ remainingUse
+ minimumSpend
+ appliedDateRange
+ bannerImage
+ isActive
+ createOne()
+ getOne()
+ find()
+ getAll()
+ updateOne()
+ use()

4.7 Component: Database

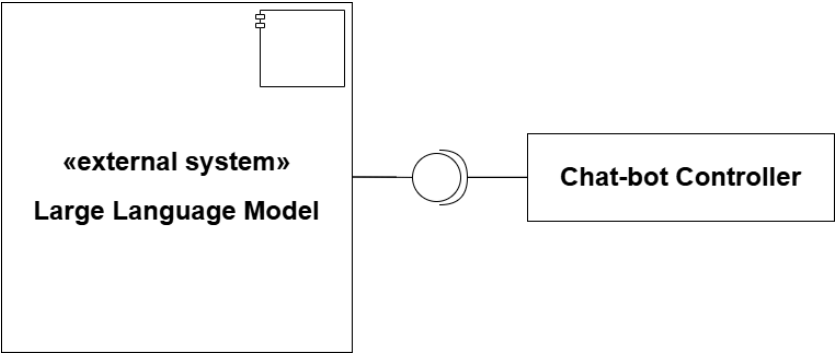


The database component serves as the persistent storage layer for the Lumiere Cinema system, designed to maintain data integrity and support efficient data operations. Its logical structure is defined by the Entity-Relationship

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

Diagram (ERD), which illustrates the primary data entities and their interconnections. This logical view of the database, as shown in the ERD, provides the foundational schema for storing all critical information related to cinema management, movie scheduling, and the comprehensive online ticket booking process.

4.8 Component: Large Language Model



Purpose: The Large Language Model (LLM) component is integrated into the Lumiere Cinema system to enable natural language understanding (NLU) and generation (NLG) capabilities for the AI-based chatbot. Its primary purpose is to process user queries in natural language, understand their intent, extract relevant information, and assist in generating intelligent responses. In Lumiere Cinema context, the LLM is an external, cloud-based API service that the Lumiere Cinema back-end interacts with via HTTP/HTTPS. It is not hosted within Lumiere's direct infrastructure.

Integration Point: The Chat-bot Controller (located within the Controllers component) serves as the primary integration point with the LLM component .

Data Flow: The Chat-bot Controller receives user queries, forwards them to the LLM for analysis, and then orchestrates calls to other internal controllers (like Movie Controller or Branch Controller) based on the LLM's parsed intent .

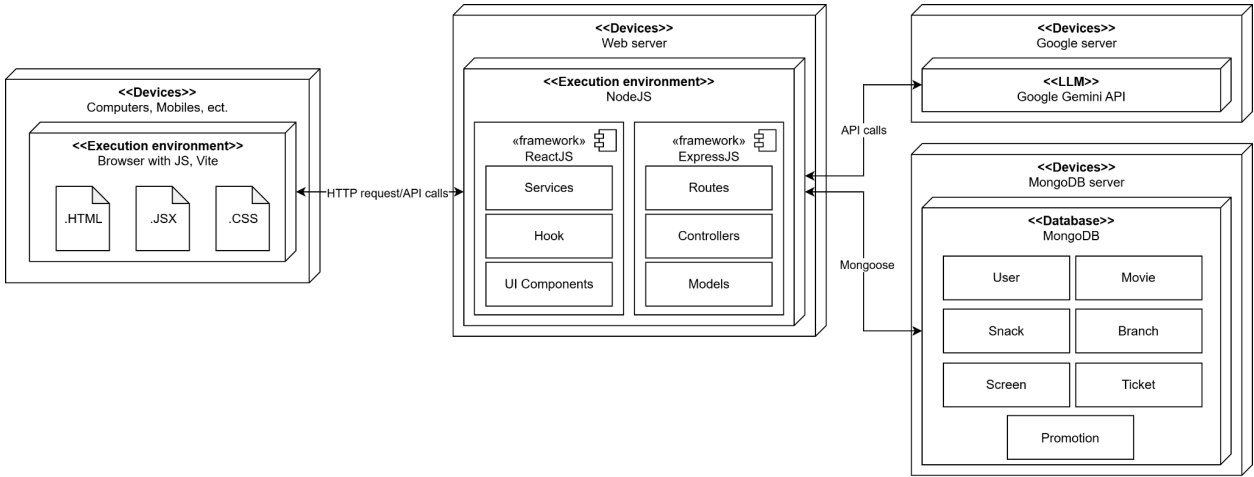
Communication Protocol: Communication between the Controllers component (specifically Chat-bot Controller) and the LLM component occurs over HTTP.

Key Technologies and Services:

- LLM Provider: (e.g., Google Gemini API).
- Internal Wrapper/Service: A dedicated service or wrapper within the Chat-bot Controller's logic handles direct API calls to the external LLM, abstracting its complexity .

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

5. Deployment



5.1 Client-side

The client-side of the Lumiere Cinema system consists of the user interface (UI) and is designed for broad accessibility and a consistent user experience across various devices and browsers.

- **Deployment:** The UI, developed using ReactJS and TailwindCSS, is accessed directly by end-users via standard web browsers. There is no dedicated client-side application server for distribution; the web application files are served by the Frontend Hosting/Web Server and rendered on the user's device.
- **Specifications:**
 - + *Supported Devices:* Desktops, Laptops (Windows, macOS), Tablets, and Smartphones (Android, iOS).
 - + *Supported Web Browsers:* Chromium-based browsers, Firefox and Apple's Safari on mobile. Any third-party services or APIs used by the client-side are also expected to support these browsers.

5.2 Server-side

The server-side encompasses the core business logic, data management, and API endpoints for the Lumiere Cinema application, built on a JavaScript runtime environment and a NoSQL database.

- **Deployment:**
 - + *Application Server:* All back-end services, hooks, routers, and controllers (including the Chat-bot Controller) are deployed on dedicated application servers, typically running Node.js with Express.js.
 - + *Database Server:* The MongoDB database can be deployed either on a dedicated local server or utilize a cloud database service like MongoDB Atlas.
 - + These application and database servers form the central backend infrastructure, handling API requests from the client-side and managing data persistence.
- **Specifications:**
 - + *Operating System:* Linux-based distributions are typically used for production environments.
 - + *Hardware/Resources:* Server specifications (CPU, RAM, Storage) will be scaled horizontally (via load balancing and multiple instances) to ensure high performance and maintain responsiveness within 3 seconds for 95% of cases, even during peak traffic.
 - + *Database Configuration:* For high availability, data integrity, and scalability, the database will be configured with replication (e.g., replica sets) and potentially sharding.

5.3 Third-Party Services Integration - Large Language Model (LLM)

The Large Language Model (LLM) component is integrated into the Lumiere Cinema system to enable intelligent, conversational capabilities within the AI-based chatbot. It enhances the user experience by enabling natural

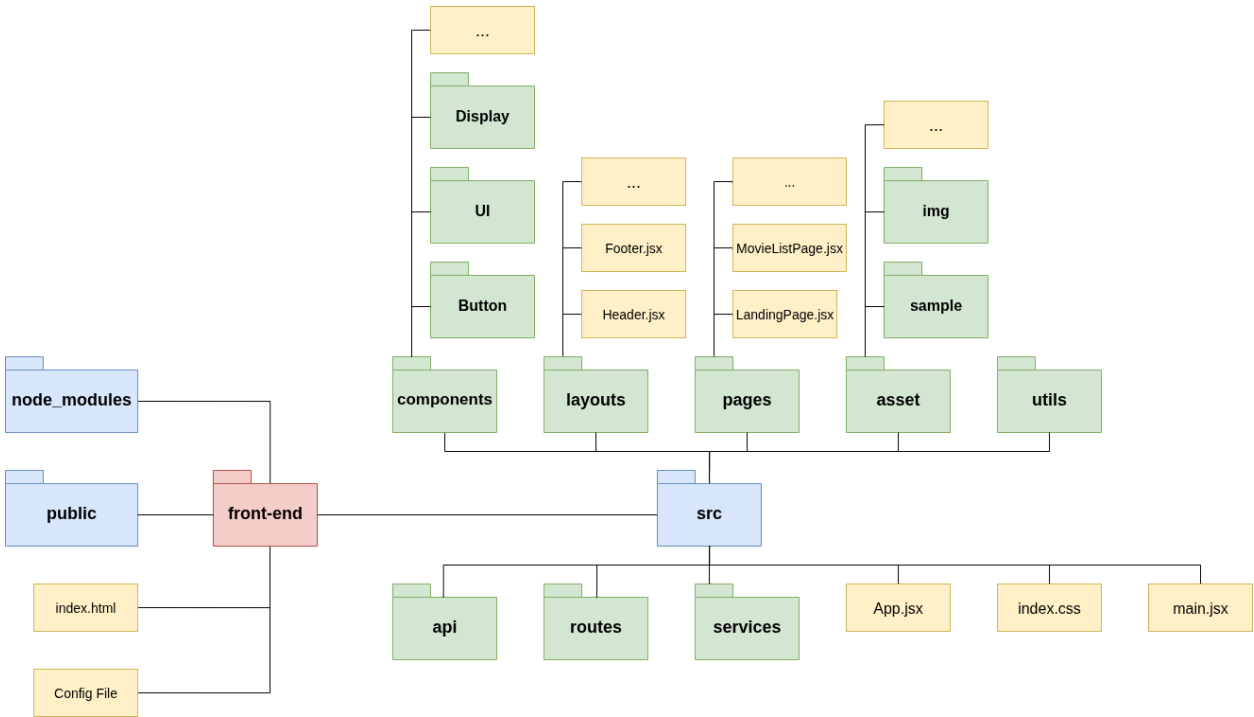
Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

language understanding (NLU) and generation (NLG), allowing users to interact with the system using free-form queries.

- **Deployment:**
 - + The LLM is not hosted on Lumiere's infrastructure but accessed as an external cloud-based service via HTTP-based API calls. The Chat-bot Controller, deployed as part of the server-side application, acts as the intermediary between user queries and the LLM service. Upon receiving user input, the Chat-bot Controller invokes the LLM API, parses its response, and routes the intent to appropriate internal controllers such as the Movie Controller or Branch Controller.
 - + A lightweight internal wrapper is implemented within the Chat-bot Controller to handle LLM API communication, abstracting authentication and formatting logic. This modular integration allows the LLM service to be replaced or extended in the future without affecting core business logic.
- **Specifications:**
 - + Provider: External LLM API service (e.g., Google Gemini API).
 - + Protocol: HTTP-based RESTful API with JSON payloads.
 - + Authentication: API keys securely stored in environment variables; not exposed to the client-side.
 - + Security: The system is designed with an abstraction layer to limit direct access and prevent sensitive data leakage. Sensitive information is not transmitted through the LLM interface.
 - + Performance: Average LLM response time is under 1 second. The system maintains overall interaction latency under 3 seconds for 95% of chatbot requests.
 - + Fallback Mechanism: If the LLM API fails (e.g., timeout, service unavailable), the chatbot responds with a default fallback message and logs the failure for analysis.
 - + Scalability: As the LLM is an external service, it benefits from the provider's scalability. The internal wrapper includes logic for retry and throttling to handle request surges.

6. Implementation View

6.1 Front-end



node_modules: Contains all third-party libraries and dependencies installed via npm/yarn. Automatically generated – should not be modified manually.

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

public: Contains static files like icons. Files here are publicly accessible and served as-is by the web server.

Config File: Stores project-wide configuration (e.g., environment variables, constants). Used to manage settings or switches that control app behavior.

src: Main source code folder for the frontend application. Includes all logic, layouts, components, and utilities of the project.

api: Stores definitions of API endpoints or base URLs.

Centralizes the configuration for making API requests.

asset: Holds shared static assets like images, and other media.

components: Contains reusable UI components such as buttons, cards, modals,.. Subdivided into smaller categories.

layouts: Stores layout-level components like headers, footers, and main structures of the page.

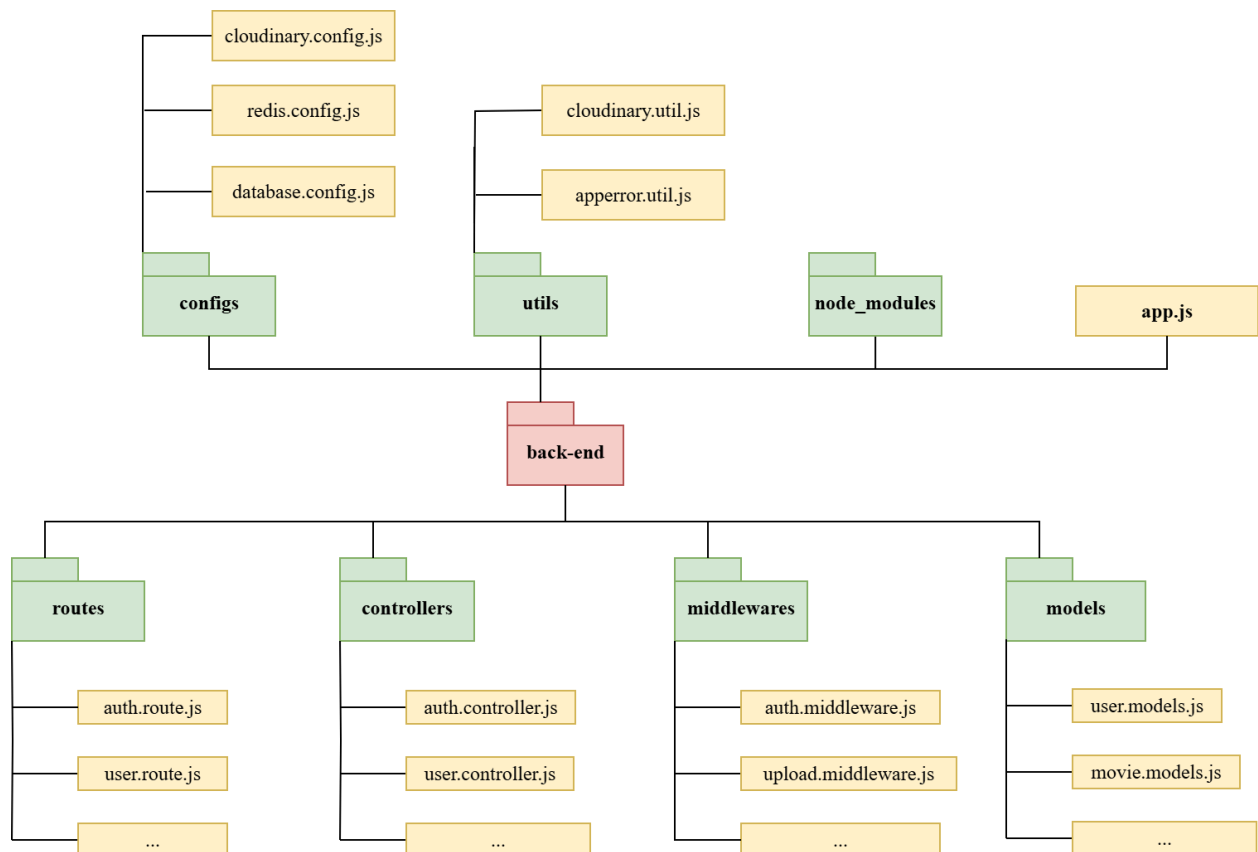
pages: Each file here represents a full page/view in the app. These are mapped to routes.

routes: Defines and manages routing logic for different pages.

services: Handles the logic for calling APIs.

utils: Contains utility/helper functions used globally.

6.2 Back-end



configs: This folder encapsulates configuration files, such as the logic for connecting to the MongoDB database (database.config.js) and setting up external services like Cloudinary, Redis.

node_modules: Contains all third-party libraries and dependencies installed via npm/yarn. Automatically generated – should not be modified manually.

controllers: Contains the application's core business logic. These files receive and process HTTP requests, interact directly with the models to perform data operations, and formulate the final response to send back to the client.

middlewares: This is used for cross-cutting concerns like validating user authentication tokens (auth.middleware.js), checking user roles for authorization, and processing file uploads(upload.middleware.js).

Lumiere Cinema	Version: 2.1
Software Architecture Document	Date: 25/07/2025
lumiere-cinema-sad-v2.1	

models: This directory contains all 13 of the Mongoose schemas that define the application's data structure, validation rules, and relationships between different data entities.

routes: Defines all the application's API endpoints. Each file in this directory maps specific URL paths and HTTP methods to the corresponding controller functions that will handle them.

utils: A collection of reusable helper functions and utility classes, to keep the codebase DRY.

app.js: The main entry point for the back-end application. This file is responsible for initializing the Express server, connecting to the database, applying global middleware, mounting the main router, and starting the server to listen for incoming requests.